

Variational Physics-informed Neural Operator (VINO) for solving partial differential equations

Mohammad Sadegh Eshaghi ^{a,*,*}, Cosmin Anitescu ^c, Manish Thombre ^{c,d},
Yizheng Wang ^{c,e}, Xiaoying Zhuang ^{a,b}, Timon Rabczuk ^{c,*}

^a Chair of Computational Science and Simulation Technology, Institute of Photonics, Department of Mathematics and Physics, Leibniz University Hannover, 30167 Hannover, Germany

^b Department of Geotechnical Engineering, College of Civil Engineering, Tongji University, Siping Road 1239, 200092 Shanghai, China

^c Institute of Structural Mechanics, Bauhaus-Universität Weimar, Germany

^d Department of Mechanical Engineering, Indian Institute of Technology Bombay, Mumbai, India

^e Department of Engineering Mechanics, Tsinghua University, Beijing, China

ARTICLE INFO

Dataset link: <https://github.com/eshaghi-ms/VINO>

Keywords:

Neural operator
Physics-informed neural network
Physics-informed neural operator
Partial differential equation
Machine learning

ABSTRACT

Solving partial differential equations (PDEs) is a required step in the simulation of natural and engineering systems. The associated computational costs significantly increase when exploring various scenarios, such as changes in initial or boundary conditions or different input configurations. This study proposes the Variational Physics-Informed Neural Operator (VINO), a deep learning method designed for solving PDEs by minimizing the energy formulation of PDEs. This framework can be trained without any labeled data, resulting in improved performance and accuracy compared to existing deep learning methods and conventional PDE solvers. By discretizing the domain into elements, the variational format allows VINO to overcome the key challenge in physics-informed neural operators, namely the efficient evaluation of the governing equations for computing the loss. Comparative results demonstrate VINO's superior performance, especially as the mesh resolution increases. As a result, this study suggests a better way to incorporate physical laws into neural operators, opening a new approach for modeling and simulating nonlinear and complex processes in science and engineering.

1. Introduction

Recent advancements in machine learning have shown potential in addressing Partial Differential Equations (PDEs) [1–8] with diverse applicability in wide applications such as mechanics [9,10] fluid mechanics [11,12], heat transfer [13], bio-engineering [14,15], materials [16,17], and finance [18,19]. Among these, a notable development is the operator learning approach for solving PDEs. Unlike traditional neural networks, which are limited to learning mappings between fixed-dimensional inputs and outputs, neural operators are designed to learn mappings between function spaces [4,5,20,21]. Neural operators are recognized as universal approximators for any continuous operator [21], equipping them to approximate a wide range of solutions for various parametric PDE families. Specifically, the solution operator maps input functions, such as initial and boundary conditions, to output solution functions [22–25].

Although the Neural Operators generally can capture complex multi-scale systems, they struggle with generalization and accurately approximating the true operator when trained only on coarse-resolution data. Furthermore, it remains challenging for

* Corresponding authors.

E-mail addresses: eshaghi.khanghah@iop.uni-hannover.de (M.S. Eshaghi), timon.rabczuk@uni-weimar.de (T. Rabczuk).

<https://doi.org/10.1016/j.cma.2025.117785>

Received 27 November 2024; Received in revised form 11 January 2025; Accepted 20 January 2025

Available online 3 February 2025

0045-7825/© 2025 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

these models to generalize effectively to new scenarios and conditions beyond those present in the training data. In many cases, generating data may be costly, unavailable, or limited to low-resolution observations [26]. The physics-informed versions of neural operators [6,7] have been introduced to resolve the shortcomings of data-driven approaches but the accuracy of current physics-informed neural operators is not sufficient unless some paired input–output data is used. Therefore, the limitation that comes from the data-driven aspect of methods and relies on the availability of data, either from existing numerical solvers or direct physical observations, still exists.

We believe that the inability of existing neural operator methods to learn PDE operators purely based on physics is due to the reliance on the strong form of governing equations, which involves higher order derivatives and is satisfied just at the collocation points in the domain. In addition, one of the major technical challenges in order to use the physical loss is to efficiently compute the derivatives of the solution [6]. Moreover, to incorporate the weak form of PDEs, the challenge of calculating the integral is also added. Therefore, we propose the **Variational Physics-Informed Neural Operator (VINO)**, which incorporates the energy form of physical laws into the networks. This form allows us to discretize the domain and use the shape functions to approximate the solution and completely resolve the differentiation and integration challenges by calculating them analytically. As a result, VINO can learn operators purely based on physical laws and with consistent hyperparameters, has better performance as the mesh size increases, and exhibits a converging behavior in finer mesh, unlike other methods such as Physics-Informed Neural Operator (PINO). In other words, we introduced a better way to incorporate the physical laws into neural operators.

It is worth noting that VINO differs from the methods based on the weak form such as [27]. In this type of method, the weak form of the governing equation is enforced by ensuring that the PDE residuals for each basis function vanish in a predefined space of test functions. In contrast, VINO optimizes the functional derived from the variational energy form. Instead of relying on multiple basis functions, VINO uses a single test function that coincides with the candidate solution itself. This means that the functional is minimized directly without iterating over test basis functions, making the computation both theoretically simpler and practically more efficient. In addition, while weak form-based methods use shape functions to directly represent the solution, VINO uses these shape functions primarily to improve the accuracy of functional evaluation over the domain. These differences allow VINO to achieve greater computational efficiency and scalability for more challenging problems. The code and data supporting this work are publicly available at <https://github.com/eshaghi-ms/VINO>.

2. Method

In this section, we introduce the PDE systems under consideration. We then provide an overview of the operator-learning and physics-informed frameworks. Finally, we present our approach in detail in the next subsections.

2.1. Problem setting

If $(\mathcal{V}, S, \mathcal{B})$ is a triplet of Banach spaces, and $\mathcal{N} : \mathcal{V} \times S \rightarrow \mathcal{B}$ is a linear or nonlinear differential operator, we consider parametric PDEs taking the form

$$\begin{aligned}\mathcal{N}(p, s) &= 0, & \text{in } \Omega \subset \mathbb{R}^d \\ s &= \bar{s}, & \text{in } \partial\Omega_D \\ \mathcal{L}(s) &= \bar{t}, & \text{in } \partial\Omega_N\end{aligned}\tag{1}$$

where $p \in \mathcal{P} \subset \mathcal{V}$ is the PDE parameters (i.e. coefficient or input functions), $s \in S$ is the corresponding unknown solutions, Ω is a bounded domain and $\mathcal{L}$ is a differential operator. Typically, the functions $\bar{s}$ and $\bar{t}$ serve as fixed boundary conditions, but they may also be included as a parameter. Then, we can consider the solution operator $\mathcal{G} : \mathcal{P} \rightarrow S$ which maps the parameters to the solutions $p \mapsto s$.

2.1.1. Solving PDEs in strong and variational form using PINNs

This section discusses the problem of approximating a solution $s^\dagger$ for a given instance $p^\dagger$ using a solution operator $\mathcal{G}^\dagger$ defined by Eq. (1). The task involves finding an approximation to the true solution by implementing ML-enhanced conventional solvers [28–30] as well as neural network-based solvers such as the PINNs and Deep Galerkin Method [1,31,32]. These PINN-type approaches use a neural network s_θ with parameters θ to approximate the solution function $s^\dagger$. The parameters θ are derived by minimizing the loss function based on the PDE strong form with exact derivatives computed utilizing the AD method. In other words, the physics-informed loss is defined by minimizing the left-hand side of Eq. (1) in the squared norm of $\mathcal{B}$, giving the loss function:

$$\mathcal{L}_{\text{pde}}(p^\dagger, s_\theta) = \alpha \mathcal{L}_{\text{system}} + \beta \mathcal{L}_{\text{BC}} = \alpha \left\| \mathcal{N}(p^\dagger, s_\theta) \right\|_{L^2(\Omega)}^2 + \beta \left\| s_\theta|_{\partial\Omega} - \bar{s} \right\|_{L^2(\partial\Omega)}^2,\tag{2}$$

where $\alpha, \beta > 0$ are hyperparameters. In this paper, the notation $\dagger$ indicates that the corresponding quantity is fixed and specified. PINNs offer the advantage of incorporating physics directly into the learning process and the universal approximability of neural networks but, in practice, PINNs often struggle to solve complex PDEs, particularly when the solution contains multi-scale features of high-frequency terms [33], or nonlinear relationships of hyperbolic PDEs, such as the problem of immiscible two-phase fluid transport in porous media [34], particularly in regions with shockwaves. whereas diffusion term in parabolic PDEs ensures improved data estimation [35].

In addition, for problems in which a variational principle exists, a functional Π can be derived in the form of

$$\Pi(p, s) = \int_{\Omega} \mathcal{F}\left(p, s, \frac{ds}{dx}, \dots\right) dx + \int_{\partial\Omega} \mathcal{E}\left(p, s, \frac{ds}{dx}, \dots\right) dx\tag{3}$$

where $\mathcal{F}$ and $\mathcal{E}$ are specified differential operators and the solution to the continuous problem is a function s which makes Π stationary. For instance, this concept is equivalent to the principle of least work in mechanics, which posits that the solution to a static mechanics problem results in the total energy — defined as the strain energy minus the external work performed by external forces at the boundary and body forces — reaching a minimum. Additionally, this principle mirrors Fermat's principle in optics (also known as the principle of least time), which asserts that the path taken by a ray of light between two points is the one that can be traversed in the shortest time. For example, the loss function in mechanical problems is formulated as [3]

$$\mathcal{L}_{\text{pde}}(p^\dagger, s) = \frac{1}{2} \int_{\Omega} \varepsilon(u_\theta) : \mathbb{C} : \varepsilon(u_\theta) d\Omega - \left(\int_{\Omega} f \cdot u_\theta d\Omega + \int_{\partial\Omega_N} \bar{t} \cdot u_\theta d\Gamma \right), \quad (4)$$

where ε denotes strain tensor, which in turn depends on the displacement u_θ predicted by a neural network with parameters θ as solution s , and constitutive elastic matrix $\mathbb{C}$, prescribed boundary forces $\bar{t}$ and body force f serve as the fixed PDE parameters $p^\dagger$.

Similar to the PINN loss function in strong form, Eq. (2), the variational energy-based PINN (VEPINN) loss function, Eq. (4) is also defined for a set of specific PDE parameters. However, there are two key differences:

- The PINN loss function, in essence, represents a set of equations that the networks attempt to satisfy at the collocation points, with the exact solution satisfying all these equations simultaneously. In contrast, the VEPINN loss function is not tied to direct equations; instead, it is a functional that the exact solution minimizes. The closer the network's predicted solution is to minimizing this functional, the closer it is to the exact solution.
- While the PINN loss function is designed to reduce the squared error (i.e., the norm) at the collocation points, with a lower limit of zero, the VEPINN loss function has no defined lower bound and can take on negative values. In other words, in physical problems, the PINN loss function represents an error and can be dimensionless. Conversely, the VEPINN loss function is not an error; it has physical dimensions and meaning. For example, in mechanical problems, it represents potential energy.

VEPINN necessitates fewer hyperparameters compared to the strong form because it incorporates the Neumann boundary condition in the physical part of the loss function and does not need to define tuning parameters in the loss function. In addition, VEPINN achieves greater computational efficiency and accuracy due to its lower derivative order [36]. However, VEPINN is dependent on the chosen integration scheme for loss calculation, which must be accurate for effective numerical integration [37,38], and it lacks generality since not all PDEs possess a corresponding variational form [39]. Current research primarily concentrates on the strong form of PINN, with relatively little focus on the variational form. The strong form involves numerous hyperparameters, particularly a penalty factor or the multiplication of coordinates to meet specific geometric boundary conditions [40]. Additionally, both PINNs and VEPINNs solve PDEs for only a single set of parameters at a time and when the parameters change, the model requires retraining, further increasing its inefficiency [9]. In this study, we aim to improve efficiency by combining neural operators and PINNs in the variational format.

2.1.2. Learning the solution operator via neural operators

An alternative approach is to learn the solution operator $\mathcal{G}$. Given a PDE described by Eq. (1) and its corresponding solution operator $\mathcal{G}$, a neural operator $\mathcal{G}_\theta$ with parameters θ can be used as a surrogate model to approximate $\mathcal{G}$. Typically, we assume that a data set $\{p_j, s_j\}_{j=1}^N$ is available, where $\mathcal{G}(p_j) = s_j$ and $p_j \sim \mu$ are independent and identically distributed (i.i.d.) samples from a distribution μ supported on $\mathcal{P}$. In this scenario, optimizing the solution operator involves minimizing the empirical data loss for a given data pair:

$$\mathcal{L}_{\text{data}}(s, \mathcal{G}_\theta(p)) = \|s - \mathcal{G}_\theta(p)\|_S^2 = \int_{\Omega} |s(x) - \mathcal{G}_\theta(p)(x)|^2 dx \quad (5)$$

The neural operator has been developed as an extension of conventional deep neural networks, to facilitate the handling of operators. The approximation of nonlinear operators is achieved by composing a linear integral operator, denoted by $\mathcal{K}$, with a pointwise nonlinear activation function, denoted by σ . The neural operator, designated as $\mathcal{G}_\theta$, is defined as follows:

$$\mathcal{G}_\theta := \mathcal{Q} \circ (\mathcal{W}_L + \mathcal{K}_L) \circ \dots \circ \sigma(\mathcal{W}_1 + \mathcal{K}_1) \circ \mathcal{P}, \quad (6)$$

wherein $\mathcal{P}$ and $\mathcal{Q}$ are pointwise operators parameterized by neural networks. In particular, the operator $\mathcal{P}$ maps from the space of dimension d_p to the space of dimension d_1 , while the operator $\mathcal{Q}$ maps from the space of dimension d_L to the space of dimension d_u . The operator $\mathcal{P}$ elevates the lower-dimensional input function to a higher-dimensional space, whereas $\mathcal{Q}$ reduces the high-dimensional output function to a lower-dimensional space.

The model stacks L layers, each of the form $\sigma(\mathcal{W}_l + \mathcal{K}_l)$, where $\mathcal{W}$ are pointwise linear operators represented as matrices $\mathcal{W}_l \in \mathbb{R}^{d_{l+1} \times d_l}$ and the kernel operators, denoted by $\mathcal{K}_l : \{\Omega \rightarrow \mathbb{R}^{d_l}\} \rightarrow \{\Omega \rightarrow \mathbb{R}^{d_{l+1}}\}$, are integral operators. The activation functions σ are fixed and the parameters, θ include all the parameters of $\mathcal{P}$, $\mathcal{Q}$, $\mathcal{W}_l$, and $\mathcal{K}_l$. For more details, we refer to [20].

As with supervised learning in computer vision and language processing, operator learning relies heavily on data, necessitating the availability of training and testing points that adhere to the same distribution. However, neural operator models like FNO have difficulty generalizing across different coefficients or geometries. Furthermore, gathering representative data is particularly challenging for complex PDEs with slow or non-existent solvers. Additionally, FNO models do not incorporate equation knowledge, which prevents them from fully closing the generalization gap and limits their broader application [41]. The methods of the physics-informed neural operator (PINO) [6] and physics-informed DeepOnets [7] are introduced to overcome these challenges, but the reliance on data and associated issues still persist. We introduce the VINO method to address these problems in the following

methods and demonstrate that dependence on data can be overcome.

VINO utilizes the FNO as its backbone structure. At its core, VINO applies a sequence of Fourier transforms and spectral operations to represent and encode the input function. The process begins by lifting the input data (defined on a spatial domain) into a higher-dimensional latent space through a linear transformation, which prepares it for subsequent spectral manipulation. Its architecture uses the Fourier transform to map the spatial representation of the function to the frequency domain, where convolution operations can be efficiently performed as element-wise multiplications. After modifying the spectral representation using learnable filters (parameterized weights defined over frequency bands), an inverse Fourier transform is applied to bring the modified function back into the spatial domain. The network concludes with a projection step, where the latent representation is translated back into the space of the original function, yielding the final solution. In the following section, we discuss the ways in which VINO differs from existing methods and demonstrate its potential applications and advantages in addressing various challenges.

2.2. Learning the solution operator via Variational Physics-Informed Neural Operator (VINO)

The limitation of data-dependence approaches, including data-driven and physics-informed neural operators, is that the solution requires a set of paired datasets, which can be challenging to gather or generate, particularly in engineering applications where data acquisition is often prohibitively expensive. With motivation from Deep Ritz [2] and DEM [3] we used the functional minimization in the variational format of PDE to train the network, obviating the necessity for a paired dataset. Although using the variational form as a loss function is mentioned in the PINO paper for solving the Darcy equation, the authors abandoned it, citing the superior performance of the strong form [6]. However, the variational form allows for domain discretization, which we inspired by the Finite Element Method, resulting in a notable enhancement. The most significant improvement involves removing the necessity of a dataset and enhancing the method's robustness to resolution changes.

Consider again the functional presented in Eq. (3). We use it directly as a loss function, so the neural network $\mathcal{G}_\theta$ tries to minimize it across all training function inputs:

$$\mathcal{L}_{\text{pde}}(\theta) = \frac{1}{n} \sum_{i=1}^n \int_{\Omega} \mathcal{F} \left(p^{(i)}, \mathcal{G}_\theta(p^{(i)}), \frac{d\mathcal{G}_\theta(p^{(i)})}{dx}, \dots \right) d\Omega + \int_{\partial\Omega} \mathcal{E} \left(p^{(i)}, \mathcal{G}_\theta(p^{(i)}), \frac{d\mathcal{G}_\theta(p^{(i)})}{dx}, \dots \right) d\Omega \quad (7)$$

The main bottleneck in deriving this loss function is the computation of the numerical integration. Various methods, such as trapezoidal, Simpson's, and Monte Carlo, have been used in the literature, but none have proven sufficient for neural operators. Furthermore, a significant technical challenge lies in efficiently computing derivatives for neural operators [6]. Numerical differentiation methods, such as finite differences, are fast and memory-efficient but encounter challenges. On one hand, they require a fine-resolution grid, while on the other, increasing the resolution increases the computational cost and may lead to numerical instabilities. To deal with these challenges, inspired by the finite element method, we can divide the domain into smaller elements and evaluate analytically the integrals individually over each element by noting

$$\mathcal{L}_{\text{pde}}(\theta) = \frac{1}{n} \sum_{i=1}^n \sum_e \int_{\Omega_e} \mathcal{F} \left(p^{(i)}, \mathcal{G}_\theta(p^{(i)}), \frac{d\mathcal{G}_\theta(p^{(i)})}{dx}, \dots \right) d\Omega_e + \int_{\partial\Omega_e} \mathcal{E} \left(p^{(i)}, \mathcal{G}_\theta(p^{(i)}), \frac{d\mathcal{G}_\theta(p^{(i)})}{dx}, \dots \right) d\Omega_e \quad (8)$$

In the following, for clarification, we derive the equations for a rectangle element with four nodes, but the process is the same for other types of elements. The rectangular element considered has side length of $2a$ and $2b$ in the x -direction and y -direction, respectively. It is convenient to use a local Cartesian system x' , y' defined by

$$x' = x - x_0 \quad \text{and} \quad y' = y - y_0 \quad (9)$$

where

$$x_0 = \frac{1}{4} \sum_{a=1}^4 x_a \quad \text{and} \quad y_0 = \frac{1}{4} \sum_{a=1}^4 y_a \quad (10)$$

in which x_0 , y_0 are located at the center of the rectangle and x_a , y_a are coordinates of the nodes. To construct approximating functions $\hat{\phi}$ for each of the solution components, denoted by ϕ , we have chosen linear shape function to ensure interelement C_0 continuity. For considering higher-order shape function one can refer to Appendix A. Thus, a suitable shape function is given by

$$\hat{\phi}^e = \alpha_1 + x' \alpha_2 + y' \alpha_3 + x' y' \alpha_4 \quad (11)$$

The coefficients α_a may be obtained by expressing Eq. (11) at each node giving

$$\begin{bmatrix} \tilde{\phi}_1^e \\ \tilde{\phi}_2^e \\ \tilde{\phi}_3^e \\ \tilde{\phi}_4^e \end{bmatrix} = \begin{bmatrix} 1 & -a & -b & ab \\ 1 & a & -b & -ab \\ 1 & a & b & ab \\ 1 & -a & b & -ab \end{bmatrix} \begin{bmatrix} \alpha_1 \\ \alpha_2 \\ \alpha_3 \\ \alpha_4 \end{bmatrix} \quad (12)$$

We can solve for α_a in terms of the nodal values to obtain finally

$$\begin{aligned}\hat{\phi}_e = & \frac{1}{4} \left(1 - \frac{x'}{a}\right) \left(1 - \frac{y'}{b}\right) \tilde{\phi}_1^e + \frac{1}{4} \left(1 + \frac{x'}{a}\right) \left(1 - \frac{y'}{b}\right) \tilde{\phi}_2^e \\ & + \frac{1}{4} \left(1 + \frac{x'}{a}\right) \left(1 + \frac{y'}{b}\right) \tilde{\phi}_3^e + \frac{1}{4} \left(1 - \frac{x'}{a}\right) \left(1 + \frac{y'}{b}\right) \tilde{\phi}_4^e,\end{aligned}\quad (13)$$

From Eq. (13) we can define the four shape function as follows:

$$\begin{aligned}N_1 = & \frac{1}{4} \left(1 - \frac{x'}{a}\right) \left(1 - \frac{y'}{b}\right), \quad N_2 = \frac{1}{4} \left(1 + \frac{x'}{a}\right) \left(1 - \frac{y'}{b}\right) \\ N_3 = & \frac{1}{4} \left(1 + \frac{x'}{a}\right) \left(1 + \frac{y'}{b}\right), \quad N_4 = \frac{1}{4} \left(1 - \frac{x'}{a}\right) \left(1 + \frac{y'}{b}\right)\end{aligned}\quad (14)$$

As the shape functions in Eq. (14) are similar for all solution components, we can write

$$\mathcal{G}_\theta^e(p) = s^e \approx \hat{s}^e = \sum_{a=1}^4 N_a \tilde{s}_a = \mathbf{N} \tilde{\mathbf{s}} \quad (15)$$

Therefore, we can calculate the derivative and integral analytically, thus solving the difficulties and challenges mentioned above. For more clarification, we derive the equation for Poisson equation in the form of

$$-\nabla^2 \phi = f \quad \text{in } \Omega \quad (16)$$

where ∇^2 is the Laplace operator, ϕ is the unknown function, and f is a known source term.

The equivalent minimization problem seeks to minimize the following functional:

$$\Pi(\phi) = \frac{1}{2} \int_{\Omega} |\nabla \phi|^2 d\Omega - \int_{\Omega} f \phi d\Omega \quad (17)$$

So the loss function, after the discretization of the domain, is defined as

$$\mathcal{L}_{\text{pde}}(\theta) = \frac{1}{n} \sum_{i=1}^n \sum_e \int_{\Omega_e} \left(\frac{1}{2} |\nabla \phi|^2 - f \phi \right) d\Omega_e \quad (18)$$

Since $\phi \approx N \tilde{\phi}$, where:

$$N = [N_1, N_2, N_3, N_4], \quad \tilde{\phi} = [\phi_1, \phi_2, \phi_3, \phi_4]^T. \quad (19)$$

Thus, the gradient of ϕ is given by:

$$\nabla \phi = B \tilde{\phi}, \quad (20)$$

where B is the gradient matrix of the shape functions:

$$B = \begin{bmatrix} \partial N_1 / \partial x & \partial N_2 / \partial x & \partial N_3 / \partial x & \partial N_4 / \partial x \\ \partial N_1 / \partial y & \partial N_2 / \partial y & \partial N_3 / \partial y & \partial N_4 / \partial y \end{bmatrix}. \quad (21)$$

Therefore, the magnitude squared of the gradient is:

$$|\nabla \phi|^2 = (\nabla \phi) \cdot (\nabla \phi) = (B \tilde{\phi})^T (B \tilde{\phi}) = \tilde{\phi}^T B^T B \tilde{\phi}. \quad (22)$$

As a result:

$$\frac{1}{2} \int_{\Omega_e} |\nabla \phi|^2 d\Omega_e = \frac{1}{2} \tilde{\phi}^T \left(\int_{\Omega_e} B^T B d\Omega_e \right) \tilde{\phi} = \frac{1}{2} \tilde{\phi}^T K^e \tilde{\phi}, \quad (23)$$

where $K^e = \int_{\Omega_e} B^T B d\Omega_e$ is a scalar quantity and the next term becomes:

$$\int_{\Omega_e} f \phi d\Omega_e = \tilde{\phi}^T \left(\int_{\Omega_e} N^T N d\Omega_e \right) \tilde{f} = \tilde{\phi}^T M^e \tilde{f}, \quad (24)$$

where $M^e = \int_{\Omega_e} N^T N d\Omega_e$ is also a scalar quantity.

Thus, the final expression, after calculating the derivatives and integral, for functional $\Pi(\phi)$ is:

$$\Pi(\phi) = \frac{1}{n} \sum_{i=1}^n \sum_e \left[\frac{1}{2} \tilde{\phi}^T K \tilde{\phi} - \tilde{\phi}^T M \tilde{f} \right]. \quad (25)$$

Therefore, to approximate the solution operator of the Poisson equation, VINO only needs to minimize Eq. (25) without calculating the derivatives and integral since they have already been calculated analytically inside each element. Note that the

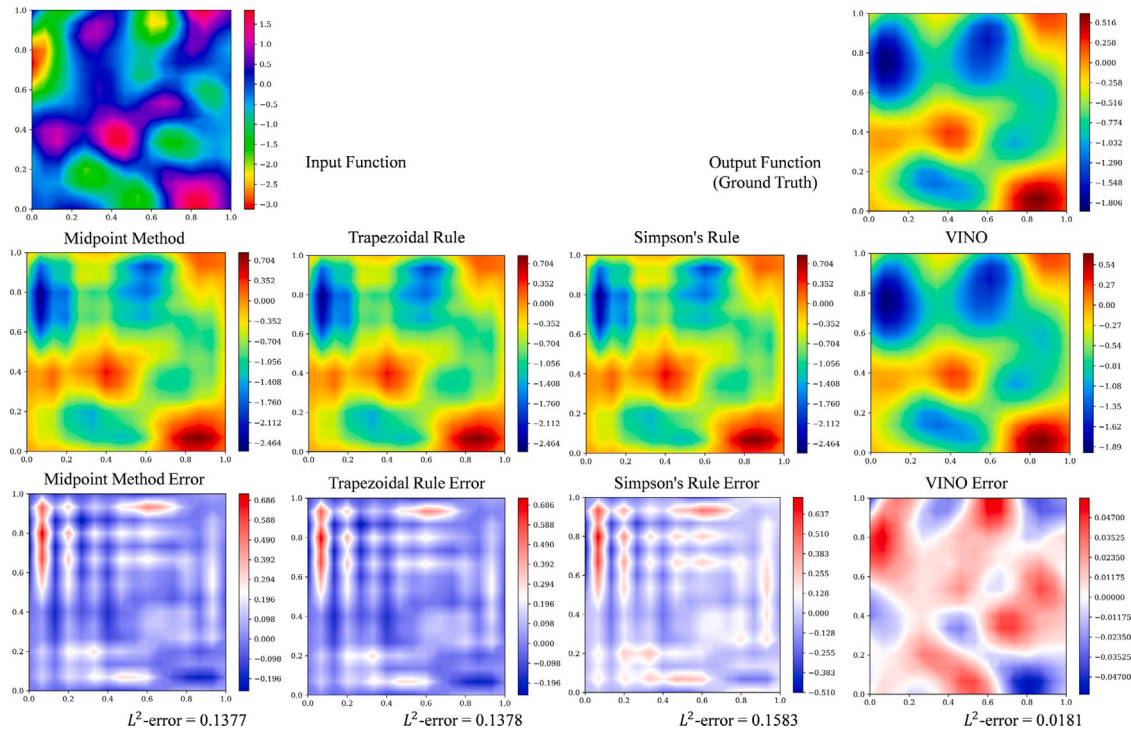


Fig. 1. Comparison of the performance of different common integral methods in the literature, and VINO for Poisson Equation with a 16×16 grid. The relative L^2 errors are indicated for each model.

result of the Eq. (25) in the whole domain is simply the summation of it over the elements and there is no concept like assembling as there is in finite element methods. The same process can be considered for other types of PDEs as well as other types of elements, which we do not mention for brevity.

To investigate the efficiency of the discretization in VINO, we compare the performance of a neural operator with the same hyperparameters and a variational loss function format. We employ common integration methods used in the literature, including the Midpoint Method, Trapezoidal Rule, and Simpson's Rule, where derivatives are computed using the Finite Differences method. These results are then compared with VINO. To emphasize the improvement offered by VINO, we use a coarse 16×16 grid.

In Fig. 1, the top row shows the input and output functions and the middle row visualizes the predictions for all the methods. The bottom row quantifies the errors spatially, illustrating how the error is distributed across the domain. These plots show the point-wise difference between each method's output and the ground truth solution. The corresponding L^2 -error for each method is noted at the bottom of these plots L^2 -error for the Midpoint Method, Trapezoidal Rule, Simpson's Rule, and VINO are 0.1377, 0.1378, 0.1583 and 0.0181 respectively. These results demonstrate that traditional numerical integration methods and finite difference approaches for derivatives are not sufficient when using a variational loss function. In contrast, the significantly lower L^2 -error for VINO highlights its superior accuracy, even on a coarse mesh.

In summary, VINO fundamentally differs from methodologies such as PINO and DeepONets in its reliance on the variational formulation of PDEs and the use of element-wise analytical derivatives. This structural difference provides unique advantages in solving large-scale nonlinear problems. VINO leverages the variational form of PDEs, minimizing an energy functional instead of directly enforcing the PDE in its strong form. This stands in contrast to PINO, which primarily imposes PDE constraints through collocation points and automatic differentiation (AD) or numerical methods such as Finite Differences to compute derivatives. Strong-form methods, like PINO, may suffer from numerical instabilities, especially for problems involving complex geometries or nonlinearities, and tend to face scalability issues as finer grids or higher-order terms increase computational demands.

In comparison, VINO's variational approach inherently operates with weaker derivative requirements, which allows it to bypass some of the stiffness and instability of strong-form enforcement. Each element's contribution to the variational energy is computed analytically using techniques inspired by the finite element method, eliminating reliance on AD or Finite Differences. This structure increases stability and computational efficiency as the problem scale increases.

DeepONets, on the other hand, are fundamentally data-driven, designed to approximate operator mappings between inputs (such as initial/boundary conditions) and solutions. While they can be adapted to incorporate physics-based constraints, they typically require large training datasets of solution snapshots [42]. Generating such high-fidelity data sets for nonlinear, high-dimensional PDEs can be computationally expensive, especially when a large parameter space must be covered. In contrast, VINO does not require labeled datasets for training; it learns directly from the underlying variational structure of the PDE, reducing dependence

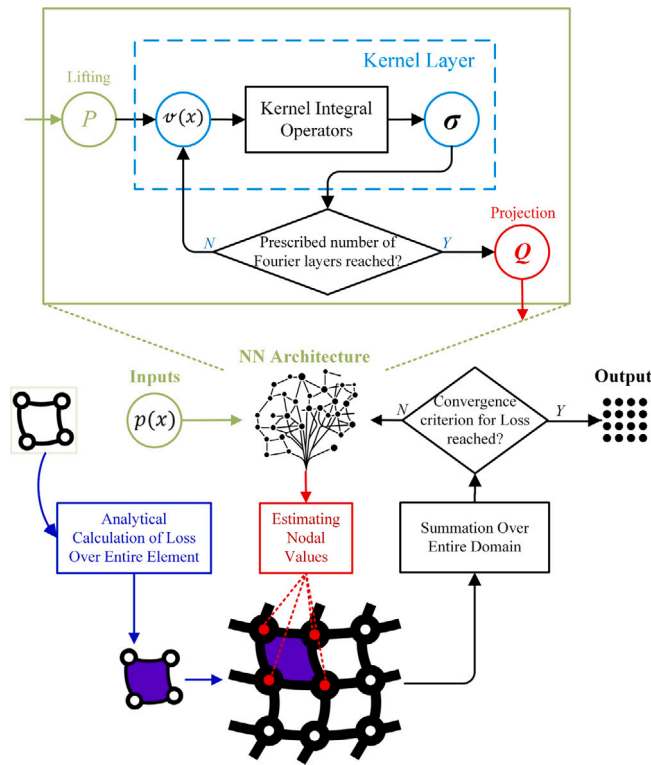


Fig. 2. Schematic Flowchart of VINO: The flowchart outlines the VINO approach, which utilizes a variational loss function derived from the functional minimization of PDEs to learn the solution operator. This method eliminates the need for paired datasets, enabling greater robustness and adaptability across varying resolutions by employing domain discretization and deriving derivatives and integral analytically over each element.

on external data.

For large-scale, nonlinear problems, VINO has significant advantages. PINO's reliance on large collocation grids or data sets can make it less efficient for high-resolution meshes, and DeepONets, while versatile, often have difficulties without sufficient paired input–output data. VINO, with its FEM-inspired variational framework, is inherently scalable; finer meshes improve energy calculation accuracy and convergence behavior, resembling classical FEM methods while retaining the expressiveness of neural networks. This combination, the focus on variational principles combined with the integration of element-wise analytical computations, makes VINO particularly suitable for large-scale problems where robust, efficient, and data-independent learning is paramount.

VINO uses FEM-based discretization of the variational form, which inherently introduces errors due to numerical integration and finite basis function approximation. However, the variational framework ensures that VINO satisfies the weak form globally over the entire domain, leading to robust convergence properties. The errors are systematic and decrease with increasing resolution, as indicated by the observed improvement in accuracy. PINO, on the other hand, enforces the PDE pointwise using collocation-based losses. This approach is highly sensitive to the choice and density of collocation points, which can lead to suboptimal domain coverage and localized errors. Moreover, the collocation method can be considered as a special type of Galerkin method where the shape functions are $\phi_m(x) = \delta(x_m)$ where $\delta(x_m)$ is such that for $x \neq x_m$, $\phi_m(x) = 0$ but $\int_{\Omega} \phi_m(x) dx = 1$ [39]. So PINO also inherits the intrinsic error of the Galerkin method.

Moreover, VINO optimizes solutions that respect the global properties of the variational form. The method approximates the solution using basis functions over the entire domain, potentially smoothing local numerical errors. PINO solves the strong form, which often requires compensating for errors at collocation points without guaranteeing smooth global representations, leading to discrepancies in solution quality. Moreover, VINO uses the weak form of PDEs, which typically requires the evaluation of lower-order derivatives. This significantly reduces computational complexity and numerical sensitivity. PINO, on the other hand, operates in the strong form and must evaluate higher-order derivatives to enforce the PDE pointwise. This not only increases the computational cost but also makes training more difficult due to the higher numerical errors and sensitivities associated with approximating these derivatives.

3. Results

The proposed Variational Physics-Informed Neural Operator framework is schematically depicted in Fig. 2. Inspired by the variational principle, VINO learns solution operators for PDEs without relying on paired datasets by minimizing the corresponding

Table 1Statistical values of relative L^2 -error and training time for different methods.

PDE Equation	FNO	PINO	VINO	PINO+data	VINO+data
Second-order anti derivative	$0.54 \pm 0.61\%$ 94.19 s	$1.83 \pm 0.97\%$ 99.59 s	$0.36 \pm 0.36\%$ 101.04 s	$0.33 \pm 0.33\%$ 109.20 s	$0.25 \pm 0.35\%$ 112.89 s
Poisson	$0.94 \pm 0.30\%$ 220.9 s	$2.63 \pm 1.92\%$ 224.1 s	$0.75 \pm 0.33\%$ 236.5 s	$1.11 \pm 0.52\%$ 226.3 s	$0.63 \pm 0.26\%$ 236.7 s
Darcy Flow	$1.49 \pm 0.58\%$ 228.2 s	$2.19 \pm 1.17\%$ 251.2 s	$0.93 \pm 0.31\%$ 244.2 s	$1.14 \pm 0.33\%$ 251.3 s	$0.98 \pm 0.46\%$ 245.6 s

Table 2

Hyperparameters used in the training of models for different problems.

Hyperparameter	2nd AntiDeriv	Poisson	Darcy	4th-Order Eq.	Porous	Hyperelasticity	Plate & voids
Train dataset size	1000	1000	1000	1000	1000	500	1000
Test dataset size	100	100	100	100	100	50	200
Number of epochs	500	500	500	2000	1000	1000	2000
Batch size	20	100	100	10	100	50	20
Discretization size	256	64	64	64	64×32	200×50	200×200
Learning rate	0.001	0.001	0.001	0.001	0.005	0.001	0.01
Decay factor	0.5	0.5	0.5	0.5	0.5	0.5	0.5
Fourier modes	16	8	12	32	8	8	8
Number of layers	4	4	4	4	4	4	8
Fourier Layer width	64	32	32	64	32	32	32

functional of PDEs as a loss function in a discretized domain. This discretization enables VINO to integrate and differentiate over each domain element analytically, removing dependency on limiting numerical methods or the computationally intensive automatic differentiation algorithm.

In the following, we illustrate the effectiveness of VINO over two other methods, Fourier Neural Operator (FNO) and Physics-Informed Neural Operator (PINO) on three benchmark examples and demonstrate its ability to solve more complicated cases across three further examples. The hyperparameters for the reported results are provided in Table 2, unless explicitly mentioned otherwise. A summary of methods' performance in the benchmark examples is presented in Table 1, and, graphically in Fig. 3a. In addition, we investigate the effect of mesh-grid size on the accuracy of different methods for solving the Darcy flow equation (for more detail refer to Section 3.1.3) in Fig. 3b. All methods are trained with consistent hyperparameters across different mesh sizes and each boxplot illustrates the distribution of errors for the corresponding mesh size. FNO (blue boxplots) demonstrates relatively consistent performance with a slight upward trend across mesh sizes. The median error shows a modest increase as the mesh size grows, indicating that FNO can effectively manage the challenges posed by finer meshes. In addition, the overall variability (as shown by the whiskers and outliers) remains relatively stable as the mesh resolution increases.

For PINO (green boxplots), the relative L_2 error initially decreases with increasing mesh size, reaching its lowest median error around the 32×32 mesh. However, beyond this mesh size, the error begins to rise, particularly at 98×98 and 128×128 . Additionally, the variability increases with larger mesh sizes, as evidenced by a wider range and more noticeable outliers. The high error at smaller grid sizes can be due to too few network parameters or insufficient accuracy in numerical floating point operations. As the grid size increases, PINO needs more iterations and more network parameters to reach the same accuracy, resulting in the 32×32 mesh yielding the best performance with constant hyperparameters. When the resolution is initially increased, more collocation points are introduced into the domain, allowing PINOs to better capture the solution by resolving finer features of the PDE. This reduces the error as the model has more information about the problem domain. As the resolution continues to increase, PINOs must balance more collocation points, which makes the optimization problem more challenging. Moreover, high resolution exacerbates numerical challenges, especially in enforcing pointwise conditions in the strong form. The gradients of the loss function can become unstable, leading to convergence issues or oscillatory behavior in the solution, which manifests as an increase in error.

For VINO (orange boxplots) in Fig. 3b, the median relative error remains low overall, demonstrating a clear improvement in accuracy as the mesh size increases. The median relative L_2 error consistently decreases with finer mesh resolutions. Furthermore, variability in error diminishes for smaller meshes, indicating that VINO achieves more reliable accuracy improvements with increasing mesh size. Consequently, these results exhibit a converging behavior for VINO in finer mesh, with constant hyperparameters, reminiscent of the finite element method.

3.1. Comparative numerical results

To illustrate the effectiveness of VINO, we present a series of numerical studies aimed at comparing with the best existing methods, including FNO and PINO. Because these methods are officially released using PyTorch frameworks, for fair comparison, we also developed our method in PyTorch for these comparative numerical experiments. However, we implemented the more efficient framework, JAX [43] for solving some practical examples in the next section.

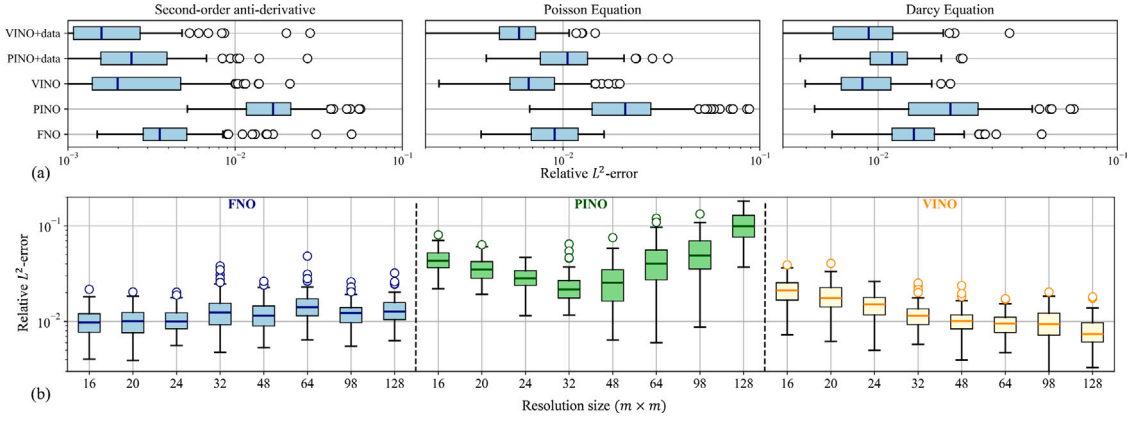


Fig. 3. Summary of the result. (a) Comparison of relative L^2 -errors for three benchmark problems across different methods: FNO, PINO, VINO, their versions combined with data. Each box plot represents the distribution of errors for each method, showing the median, interquartile range, and outliers, indicating the robustness and accuracy of each method for the given problem. Lower L^2 -error values correspond to better performance, with VINO+data generally demonstrating lower error across all benchmarks. (b) Comparison of the relative L_2 error for FNO, PINO, and VINO methods on the Darcy equation test dataset across different mesh sizes. The boxplots show the distribution of errors for each method, the y-axis shows the relative L_2 error on a logarithmic scale, while the x-axis represents the mesh size, ranging from 16×16 to 128×128 .

In most cases, we represent random input functions $p(x)$ using Gaussian random fields (GRF) [44] defined as

$$p(x) \sim \mathcal{GP}(a; k_l(x_1, x_2)), \quad (26)$$

where $k_l(x_1, x_2)$ is an exponential quadratic covariance kernel given by

$$k_l(x_1, x_2) = \exp\left(-\frac{\|x_1 - x_2\|^2}{2l^2}\right), \quad (27)$$

and a and l represent the average of the field and positive length scale respectively. The parameter l is used to regulate the complexity of the sampled input functions, with larger values of l typically resulting in smoother p . The numerical experiments were conducted on an NVIDIA A100-PCIE-40 GB GPU and the networks are trained with mini-batch stochastic gradient descent, optimized using the Adam algorithm [45].

3.1.1. Second-order anti-derivative

To demonstrate the capability of VINO in solving PDEs, let us first consider a simple ODE involving the second-order anti-derivative operator with the form

$$\begin{aligned} \frac{d^2 s(x)}{dx^2} + p(x) &= 0, \quad x \in [0, 1], \\ s(0) &= s(1) = 0. \end{aligned} \quad (28)$$

Given that $p(x)$ is a known ODE parameter, the goal is to learn the operator $\mathcal{G}$ mapping from $p(x)$ to the solution $s(x)$. To achieve this, we compare the results and performance of FNO, PINO, and VINO, with the operator denoted by $\mathcal{G}_\theta$ with the same set of hyperparameters. Given a collection of paired parameters and solutions $\{\mathbf{p}^{(i)}, \mathbf{s}^{(i)}\}$, where $\mathbf{s}^{(i)} = \mathcal{G}(\mathbf{p}^{(i)})$, the loss functions for FNO, $\mathcal{L}_f$ is given by:

$$\mathcal{L}_f := \|\mathcal{G} - \mathcal{G}_\theta\|_{L^2}^2 = \frac{1}{n} \sum_{i=1}^n \|\mathbf{s}^{(i)} - \mathcal{G}_\theta(\mathbf{p}^{(i)})\|_{L^2}^2, \quad (29)$$

and using the ODE itself, we have the loss function for PINO, $\mathcal{L}_p$ as following:

$$\mathcal{L}_p := \frac{1}{n} \sum_{i=1}^n \left\| \frac{d^2}{dx^2} \mathcal{G}_\theta(\mathbf{p}^{(i)})(x) + \mathbf{p}^{(i)}(x) \right\|_{L^2}^2 \quad (30)$$

where $\mathbf{p}^{(i)} = [p^{(i)}(x_1), p^{(i)}(x_2), \dots, p^{(i)}(x_m)]$ and the solutions $\mathbf{s}^{(i)} = [s^{(i)}(x_1), s^{(i)}(x_2), \dots, s^{(i)}(x_m)]$, may be obtained by solving the ODE with a numerical method.

In addition, using the variational principle, a functional Π for the current problem can be defined in the form of

$$\Pi = \int_{\Omega} \left[\frac{1}{2} \left(\frac{ds}{dx} \right)^2 - p(x) s(x) \right] d\Omega \quad (31)$$

and the solution is a function which makes Π minimum so it also may be called the energy minimization principle. Therefore, after discretization and using approximative function $\mathcal{G}_\theta^e(p) = \mathbf{N}\mathbf{s}_e$, which is discussed in details in Section 2, we can define the loss

function for VINO, $\mathcal{L}_v$ as:

$$\mathcal{L}_v := \frac{1}{n} \sum_{i=1}^n \sum_e \left[\frac{1}{2} \tilde{s}_e^{(i)T} \int_{\Omega_e} \left(\frac{dN}{dx} \right)^T \frac{dN}{dx} d\Omega_e \tilde{s}_e^{(i)} - \tilde{s}_e^{(i)T} \int_{\Omega_e} (N^T N) d\Omega_e \tilde{p}_e^{(i)} \right] \quad (32)$$

where N , $\tilde{s}_e$ and $\tilde{p}_e$ are the shape functions, nodal values of solution and parameter functions and e is the element index. The loss function also, in a simpler way, can be represented as follows:

$$\mathcal{L}_v = \frac{1}{n} \sum_{i=1}^n \sum_e \left[\frac{1}{2} \tilde{s}_e^{(i)T} \mathbf{K}^e \tilde{s}_e^{(i)} - \tilde{s}_e^{(i)T} \mathbf{M}^e \tilde{p}_e^{(i)} \right] \quad (33)$$

with element matrices $\mathbf{K}^e$ and $\mathbf{M}^e$ defined as:

$$\mathbf{K}^e := \int_{\Omega_e} \left(\frac{dN}{dx} \right)^T \frac{dN}{dx} d\Omega_e \quad (34)$$

$$\mathbf{M}^e := \int_{\Omega_e} (N^T N) d\Omega_e \quad (35)$$

Therefore, on a uniform rectangular grid, the matrices K_e and M_e are the same for all elements which reduces the computational and memory cost.

We sample $n = 1,000$ input functions $p(x)$ from a Gaussian random field (GRF) with a length scale $l = 0.1$. Additionally, we set $m = 256$ sensors $\{x_j\}_{j=1}^m$ uniformly spaced grid points in $[0, 1]$ and repeat the same procedure to generate a test dataset, which contains 100 different samples of random input functions and solutions. Finally, we evaluate and compare the performance of the three models using input functions that are not derived from the GRF, and therefore differ from both the training and test data. The corresponding solutions are obtained by analytically solving the ODE.

We train the models by optimizing the specified loss function for 1,000 iterations using gradient descent with the Adam optimizer. To see the effect of hyperparameters refer to [Appendix B](#). [Fig. 4](#) presents the results of applying the FNO, PINO, and VINO models to different types of test functions. Specifically, the figure shows the predictions of each model (in red) against the true solutions (in blue) and their point-wise errors for three distinct test cases:

- [Fig. 4a](#) — Gaussian Random Field (GRF): This row shows the performance of each model on a function sampled from the Gaussian random field, similar to the training data. The point-wise error below each plot reveals that while all models capture the overall trend of the solution, VINO exhibits a smaller error magnitude, as seen in the lower point-wise error values and the relative L^2 errors.

- [Fig. 4b](#) — Sinusoidal Function: In this case, the function is sinusoidal, differing from the GRF data used during training. The comparison between PINO and VINO reveals that VINO accurately captures both the phase and amplitude of the sine wave, while PINO struggles in some areas. Moreover, VINO shows a noticeably lower error compared to PINO, reflecting its improved ability to generalize.

- [Fig. 4c](#) — Exponential Function: The exponential function further challenges the models. Here, the VINO model again shows a marked improvement over the PINO. Its point-wise error remains more contained, indicating that VINO generalizes better even for inputs that significantly differ from the training data.

In addition, we compare the performance of the PINO and VINO when combined with data on this parametric ODE problem. The first row in [Table 1](#) summarizes a statistical comparison of the relative L_2 prediction error for the output functions for different models, and their variants combined with data over 100 examples in the test dataset.

We observe that VINO and its combination with data exhibit lower error means compared to PINO. In addition, VINO shows smaller deviations and variances compared to PINO, suggesting that they are more stable across different datasets. Therefore, the table and box plot indicate that VINO outperforms PINO. Note that the training time is approximately the same and VINO does not impose more computational demand.

3.1.2. Poisson equation

The next example involves two-dimensional Poisson equation, which is a second-order PDE with a source term $p(x)$. The equation is given by:

$$\begin{aligned} \nabla^2 s(x) + p(x) &= 0, \quad x \in \Omega = [0, 1]^2, \\ s(x) &= 0, \quad \text{in } \partial\Omega. \end{aligned} \quad (36)$$

Here, the goal is to learn the operator $\mathcal{G}$ mapping from PDE parameter $p(x)$ to the solution $s(x)$. Similar to the 1D case, we compare the performance of FNO, PINO, and VINO, denoted as $\mathcal{G}_\theta$, using identical hyperparameter settings. For a collection of input-output pairs $\{s^{(i)}, \mathbf{p}^{(i)}\}$, where $s^{(i)} = \mathcal{G}(\mathbf{p}^{(i)})$, the loss functions for FNO is same as Eq. (29), but PINO and VINO loss function, with the same procedure, is defined as

$$\mathcal{L}_p := \frac{1}{n} \sum_{i=1}^n \left\| \frac{\partial^2}{\partial x^2} \mathcal{G}_\theta(\mathbf{p}^{(i)}) + \frac{\partial^2}{\partial y^2} \mathcal{G}_\theta(\mathbf{p}^{(i)}) + \mathbf{p}^{(i)} \right\|_{L^2}^2 \quad (37)$$

$$\mathcal{L}_v := \frac{1}{n} \sum_{i=1}^n \sum_e \left[\frac{1}{2} \tilde{s}_e^{(i)T} \int_{\Omega_e} (\nabla N^T \nabla N) d\Omega_e \tilde{s}_e^{(i)} - \tilde{s}_e^{(i)T} \int_{\Omega_e} (N^T N) d\Omega_e \tilde{p}_e^{(i)} \right] \quad (38)$$

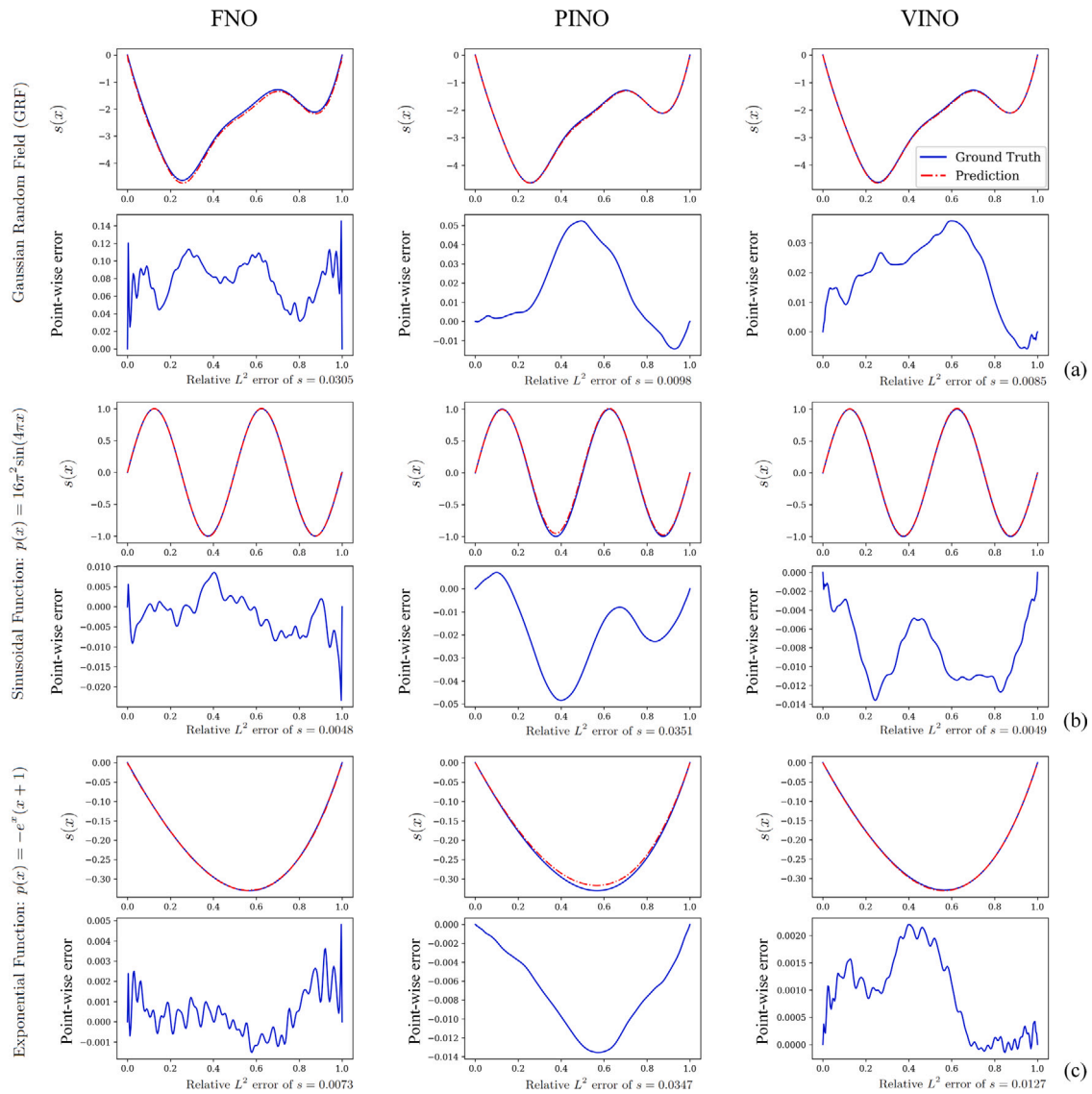


Fig. 4. Comparison of the performance of FNO, PINO, and VINO for three representative test cases: (a) Gaussian Random Field (GRF), (b) Sinusoidal function, and (c) Exponential function. The top row in each case shows the predicted solutions (red) against the true solutions (blue), while the bottom row shows the point-wise error. The relative L^2 errors are indicated for each model in each case.

In this case, $\mathbf{p}^{(i)} = [p^{(i)}(x_1), p^{(i)}(x_2), \dots, p^{(i)}(x_m)]$ and $\mathbf{s}^{(i)} = [s^{(i)}(x_1), s^{(i)}(x_2), \dots, s^{(i)}(x_m)]$, with $m = 64 \times 64$ grid points in the $[0, 1] \times [0, 1]$ domain. The input functions $p(x)$ are sampled from a Gaussian random field (GRF). We use $n = 1,000$ training samples and generate an additional test dataset of 100 different random input functions and solutions and train the models using gradient descent with the Adam optimizer.

Fig. 5 illustrates the results for a sample from the test dataset. The figure shows that VINO performs better than PINO. In addition, as shown in Table 1, the second row, the mean errors highlight that VINO, both in its standalone and data-enhanced form, consistently achieves the lowest mean L^2 error of 0.00751 and 0.00631, respectively. This indicates that VINO is more effective at solving the Poisson equation than PINO. In contrast, PINO not only has a higher mean error but also a greater deviation of 0.0192, indicating more variability in its performance across trials. Incorporating data improves the performance of both methods, with data-enhanced VINO achieving the lowest mean error and a reduced standard deviation of 0.0026. Notably, even with data augmentation, PINO's performance does not surpass that of VINO in its standalone form. As shown in the table, the training times and computational demands are very similar for all three methods.

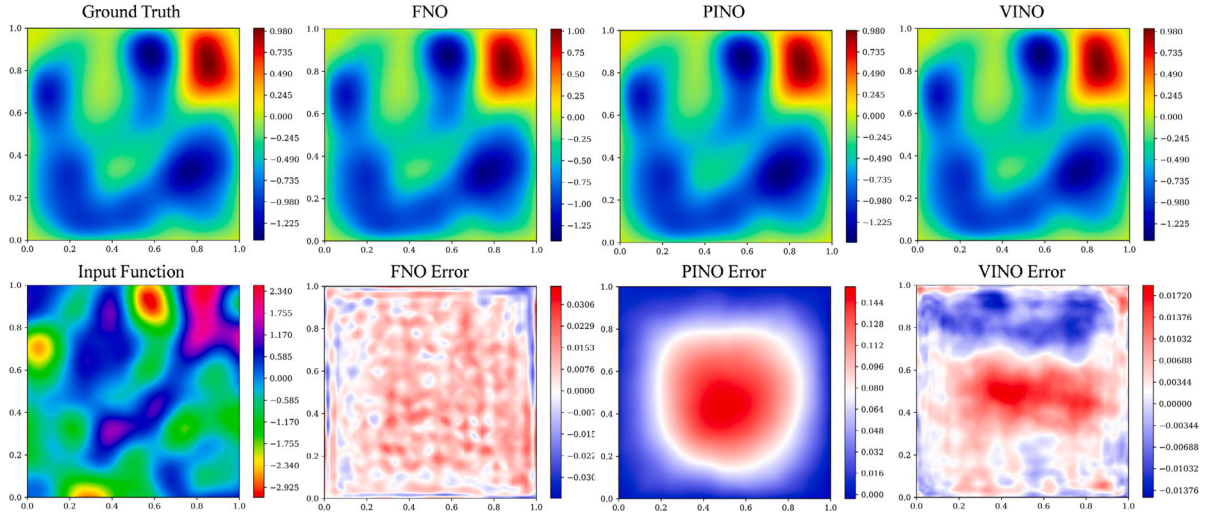


Fig. 5. Performance comparison of FNO, PINO, and VINO for a sample from test dataset.

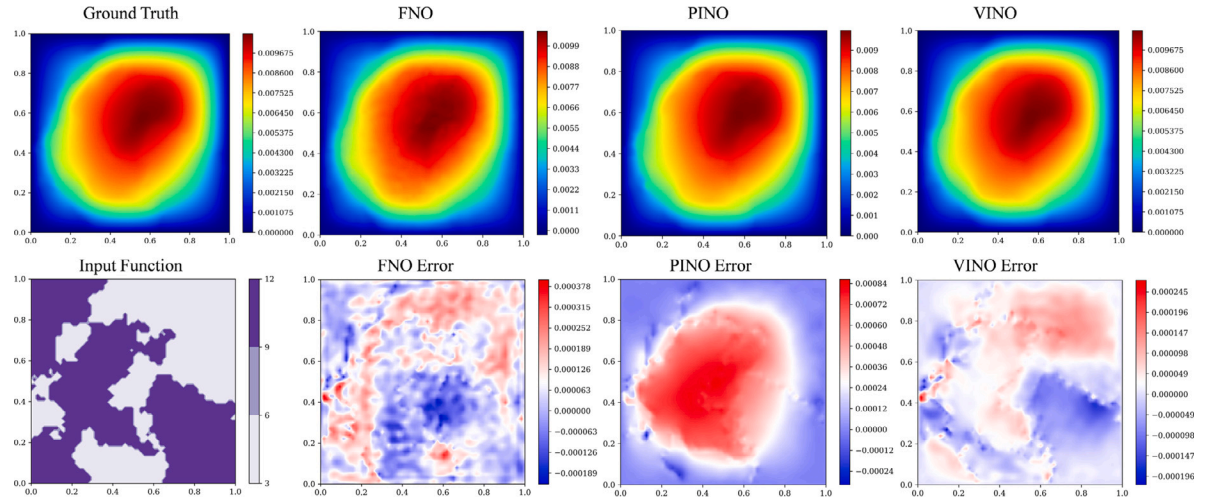


Fig. 6. Performance comparison of FNO, PINO, and VINO for a sample from the test dataset in Darcy Equation.

3.1.3. Darcy flow

As the next benchmark example, we focus on modeling two-dimensional subsurface flows through a porous medium with heterogeneous permeability fields. The high-fidelity synthetic simulation data are based on Darcy's flow, explored in several neural operator studies [4,20,46–48]. The governing differential equation for this problem is given by:

$$\begin{aligned} -\nabla \cdot (p(x)\nabla s(x)) &= f(x), \quad x \in [0, 1]^2 \\ s(x) &= 0, \quad x \in \partial[0, 1]^2 \end{aligned} \quad (39)$$

Here, $p(x)$ is a piecewise constant conductivity field in $L^\infty((0, 1)^2; \mathbb{R}_+)$, and $f(x) = 1$ is a constant source term function. The goal is to learn the operator $\mathcal{G}$ that maps the conductivity field $p(x)$ to the solution, the hydraulic head, $s(x)$, i.e., $\mathcal{G} : p \mapsto s$. Despite the linearity of the PDE, the operator $\mathcal{G}$ is nonlinear.

As typical in subsurface flow simulations, the permeability $p(x)$ is modeled as a piecewise constant function with random geometry, where the two values of $p(x)$ have a ratio of 4. Specifically, 1100 samples of $p(x)$ were generated according to the GRF distribution. Of these, 1000 samples were used for training, while the remaining were reserved for testing.

As stated in Table 1, VINO has the lowest mean error, indicating the best overall performance, while PINO has the highest mean error and the largest standard deviation, reflecting more variability in its results. The addition of data reduces both the mean error and standard deviation for PINO and VINO, improving their performance and consistency. The training time across methods is relatively consistent, with small variations.

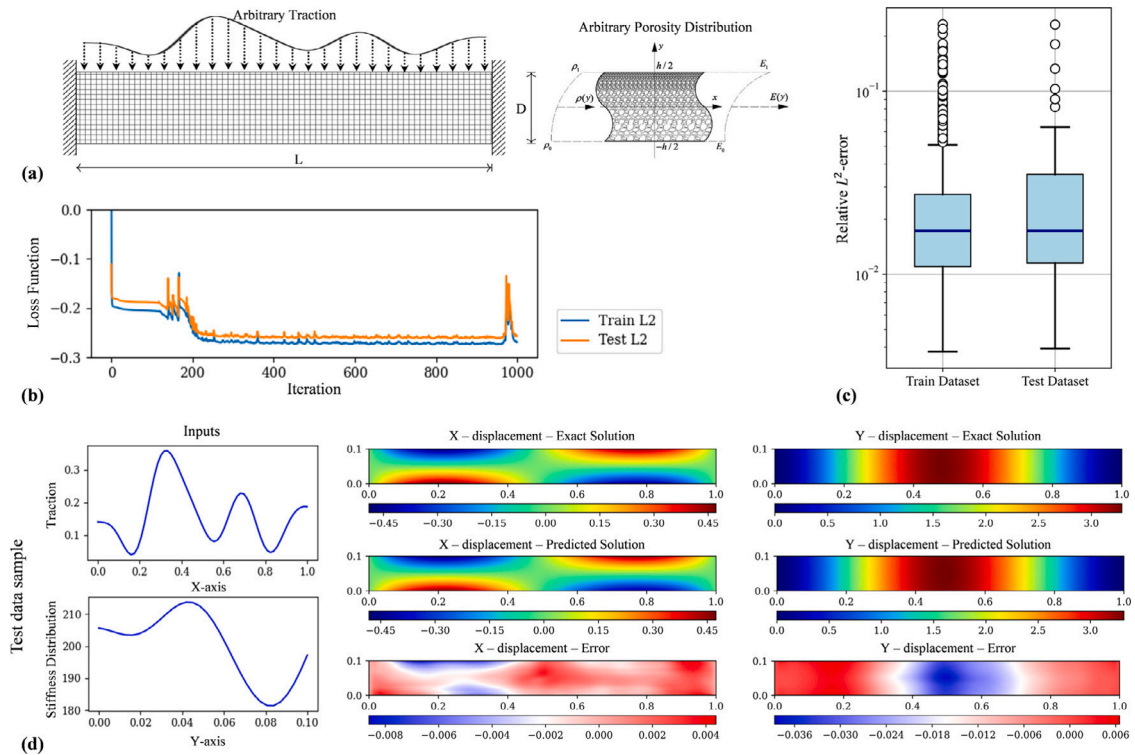


Fig. 7. Porous Material Structure. (a) Beam with Random Distributed Load and Random Porosity Distribution. (b) Loss function convergence over 1000 iterations for the train and test datasets, where both curves exhibit a rapid initial decrease followed by stabilization at lower values. (c) Box plots showing the relative L^2 -error distribution for the train and test datasets, with mean values and standard deviations. The train dataset has a mean error of 0.035 with a standard deviation of 0.087, while the test dataset has a mean error of 0.032 with a standard deviation of 0.051. (d) Performance of VINO for predicting the displacement field of the porous material beam in a random sample from train and test dataset.

The box plot in Fig. 3a further illustrates the distribution of relative L^2 -error for each method. The boxes represent the interquartile range, with the line inside the box showing the median error. PINO exhibits the widest box, indicating a greater spread in error values, while VINO and VINO+data display narrower boxes, suggesting lower and more consistent errors. Outliers are visible as circles outside the whiskers, highlighting points where the error was significantly different from the majority of the data. The box plot visually confirms the improvement in the performance of VINO in comparison with PINO. We note that, even by incorporating data into PINO, its accuracy is lower than VINO without data-driven training. In addition, Fig. 6 confirms the same result.

3.2. Practical numerical results

To demonstrate VINO's practical applicability and efficiency in addressing complex physical scenarios, we present three representative examples in this section: porous material structure, hyperelasticity, and a plate with arbitrary holes. Each example addresses distinct challenges, including material heterogeneity, large deformation, and geometric complexity, providing an evaluation of VINO's versatility and effectiveness in real-world engineering contexts. To take advantage of optimized linear algebra operations, the JAX [43] framework has been used in the following experiments.

3.2.1. Porous material structure

Porous materials are crucial in different fields like aerospace and automotive due to their ability to reduce thermal stress and maintain lightweight design resulting in unique characteristics that are not possible with homogeneous materials [49,50]. On the other hand, designing porous structures with desirable properties requires a lot of analytical and computational efforts and numerous studies in the literature explore the mechanical analysis of porous beams [51–56]. Therefore, the goal in this example is to predict their behavior based on the porosity distribution of material and distributed load.

To exemplify this, VINO has been trained to learn an operator, that maps from arbitrary traction and elasticity module distribution functions into the displacement field of a beam with depth D and length L . In other words, the networks are employed to solve a range of problems with different arbitrary traction functions and porosity distributions, shown in Fig. 7a. The beam is considered to be attached to supports at $x = 0$ and $x = L$ and sustains a distributed load on the top edge of the beam. Therefore, the governing

PDE becomes as follows:

$$\begin{aligned} \frac{\partial \sigma_{xx}}{\partial x} + \frac{\partial \sigma_{xy}}{\partial y} + f_x &= 0 \\ \frac{\partial \sigma_{yx}}{\partial x} + \frac{\partial \sigma_{yy}}{\partial y} + f_y &= 0 \\ \mathbf{u}(x, y) &= 0 \quad \text{for } x = 0, L \\ \boldsymbol{\sigma} \cdot \mathbf{n} &= \hat{\mathbf{t}}(x) \quad \text{for } y = D \end{aligned} \quad (40)$$

where f_x and f_y are body forces per unit volume and the stress components are derived based on the following constitutive relations:

$$\boldsymbol{\sigma} = E \mathbf{D} \boldsymbol{\varepsilon} \quad (41)$$

where E is the elasticity modulus and can vary over the entire domain, matrix $\mathbf{D}$, strain vector $\boldsymbol{\varepsilon}$ and stress vector $\boldsymbol{\sigma}$ are

$$\mathbf{D} = \frac{1}{1-\nu^2} \begin{bmatrix} 1 & \nu & 0 \\ \nu & 1 & 0 \\ 0 & 0 & \frac{1-\nu}{2} \end{bmatrix} \quad (42)$$

$$\boldsymbol{\sigma} = [\sigma_{xx}, \sigma_{yy}, \sigma_{xy}]^T, \quad \boldsymbol{\varepsilon} = [\varepsilon_{xx}, \varepsilon_{yy}, \varepsilon_{xy}]^T \quad (43)$$

Here, the strain components are derived by

$$\varepsilon_{xx} = \frac{\partial u}{\partial x}, \quad \varepsilon_{yy} = \frac{\partial v}{\partial y}, \quad \varepsilon_{xy} = \varepsilon_{yx} = \frac{1}{2} \left(\frac{\partial u}{\partial y} + \frac{\partial v}{\partial x} \right) \quad (44)$$

The desired operator from this PDE is $\mathcal{G} := \mathcal{L} \{ \hat{\mathbf{t}}, \hat{E} \} : \mathcal{U}^t \times \mathcal{U}^E \rightarrow \mathcal{U}$ defined to map the traction and porosity distribution to the corresponding displacement field $\{ \hat{\mathbf{t}}, \hat{E} \} \mapsto \mathbf{u}$, where $\mathcal{U}^t$ is the space of continuous real-valued functions defined on the top boundary, $\mathcal{U}^E$ is the space of continuous real-valued functions defined on $[0, L] \times [0, D]$, and $\mathcal{U}$ is the space of continuous functions with values in $\mathbb{R}^2$, representing the x - and y -displacements. We solve the problem when the domain, Ω is a rectangle with corners at $(0, 0)$ and $(1, 0.1)$, and $\nu = 1/3$ is the Poisson ratio. For the data generation and validation, the isogeometric analysis (IGA) has been used [57]. To create a dataset including the solution $\{ \hat{\mathbf{t}}^{(i)}, \hat{E}^{(i)}, \mathbf{u}^{(i)} \}$, we have used a Gaussian Random Fields (GRF) [44] with a length scale $l = 0.1$ for traction and a length scale $l = 0.025$ for porosity distribution as

$$\begin{aligned} \hat{\mathbf{t}}(x) &\sim \sigma_t \times \mathcal{GP}(0, k_1(x_1, x_2)) + \mu_t \\ \hat{E}(x) &\sim \frac{E_{\max} - E_{\min}}{\mathcal{GP}_{\max} - \mathcal{GP}_{\min}} \times (\mathcal{GP}(0, k_1(x_1, x_2)) - \mathcal{GP}_{\min}) + E_{\min} \end{aligned} \quad (45)$$

where σ_t and μ_t are expected variance and the average of traction (here $\sigma_t = 0.15$ and $\mu_t = 0.2$), $E_{\min}$ and $E_{\max}$ are expected values for minimum and maximum of elasticity modulus, which in the current example are selected randomly between 20 GPa and 380 GPa. Therefore our goal is to construct an approximation of $\mathbf{u}^{(i)} = \mathcal{G}(\hat{\mathbf{t}}^{(i)}, \hat{E}^{(i)})$ by the parametric map $\mathcal{G}_\theta : \mathcal{U}^t \times \mathcal{U}^E \rightarrow \mathcal{U}$, $\theta \in \mathbb{R}^p$.

The displacement field was computed using $n_x \times n_y$ uniformly spaced points within the domain, with $n_x = 64$ and $n_y = 32$. Throughout all layers, the GELU (Gaussian Error Linear Unit) activation function was employed [58]. The optimization process utilized the Adam optimizer, and the network underwent training on 1000 sets of GRF functions and was subsequently tested on 100 additional sets of GRF functions. The computational time taken for the training part is 376 s, but once the training is complete, solving the problem with any arbitrary function takes approximately 3 ms, compared to 1.5 s for IGA.

Figs. 7b and 7c display the convergence of the loss function, and the box plot, which illustrates the distribution of relative L^2 -error for training and test dataset. Moreover, Fig. 7d shows the result of the VINO in predicting the displacement field of a sample from training and a sample from the test dataset. The VINO result is in agreement with the ground truth solution.

3.2.2. Hyperelasticity

The next practical example is nonlinear large deformation hyperelasticity problems. Consider a body B composed of a homogeneous, isotropic, nonlinear hyperelastic material. The body is represented by a set of particles or material points X , bounded by the initial boundary ∂B . When a load t is applied over certain regions of this boundary (illustrated in Fig. 8a), the body undergoes deformation. The mapping of each material point from its initial configuration to the deformed or current configuration is given by $\boldsymbol{\varphi} : B \rightarrow B_t$, where $X \rightarrow \boldsymbol{\varphi}(X, t) = \mathbf{x} = \mathbf{u} + \mathbf{X}$.

We define a boundary value problem in the initial configuration, which includes the following components:

$$\begin{aligned} \nabla \cdot \mathbf{P} + \mathbf{f}_b &= \mathbf{0} \\ \mathbf{u} &= \bar{\mathbf{u}} \quad \text{on } \partial B_u \\ \mathbf{P} \cdot \mathbf{n} &= \mathbf{t} \quad \text{on } \partial B_t \end{aligned} \quad (46)$$

where $\nabla \cdot \mathbf{P}$ represents the divergence of the 1st Piola–Kirchhoff stress tensor $\mathbf{P}$ with respect to $\mathbf{X}$, and $\mathbf{f}_b$ is the body force. $\bar{\mathbf{u}}$ is the prescribed displacement on the Dirichlet boundary ∂B_u and $\mathbf{t}$ is the traction on the Neumann boundary ∂B_t , and $\mathbf{n}$ is the outward unit normal vector. The boundaries are required to satisfy $\partial B_u \cup \partial B_t = \partial B$ and $\partial B_u \cap \partial B_t = \emptyset$. The 1st Piola–Kirchhoff stress tensor $\mathbf{P}$ is connected to the deformation gradient $\mathbf{F}$, its power conjugate, defined by

$$\mathbf{F} = \nabla \boldsymbol{\varphi}(\mathbf{X}) \quad (47)$$

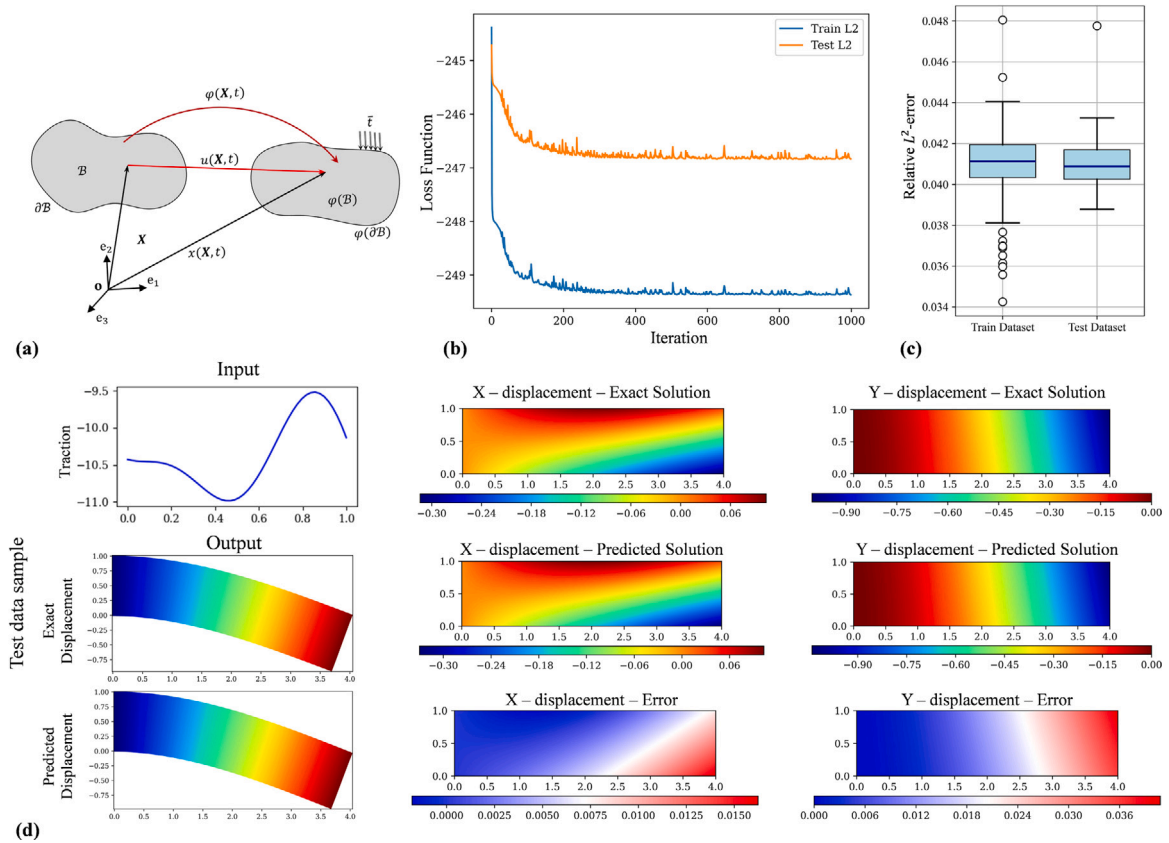


Fig. 8. Hyperelasticity. (a) Motion of body B . (b) Training/testing loss convergence over 1000 iterations. The Line plot shows the model's loss reduction and stabilization across iterations (c) Comparison of relative L^2 -errors of VINO for train and test datasets. The box plot highlights error consistency between datasets (d) Performance of VINO for predicting the displacement field of the hyperelasticity problem in a random sample from test dataset.

through a constitutive relation, specified by

$$\mathbf{P} = \frac{\partial \Psi}{\partial \mathbf{F}} \quad (48)$$

where Ψ represents the strain energy density function of the material. Considering the procedure is the same for other hyperelastic models, we examined Mooney–Rivlin hyperelastic models for Ψ

$$\Psi(I_1, I_2, J) = c(J - 1)^2 - d \log(J) + c_1(I_1 - 3) + c_2(I_2 - 3) \quad (49)$$

where $I_1 = \text{trace}(\mathbf{C})$, $I_2 = \frac{1}{2}(\text{trace}(\mathbf{C})^2 - \text{trace}(\mathbf{C} \cdot \mathbf{C}))$, and $J = \sqrt{\det(\mathbf{C})}$. The constants c , c_1 , c_2 , and d (where $d = 2(c_1 + 2c_2)$) characterize the material's properties, assuming a stress-free reference configuration and $\mathbf{C} = \mathbf{F}^T \cdot \mathbf{F}$ is the right Cauchy–Green tensor.

In finite element analysis, solving the balance equation with the appropriate boundary conditions involves transforming the problem's strong form to a weak form. Using the principle of virtual displacements, linear momentum can be derived in the initial configuration. However, there exists a strain energy, which is the elastic energy stored in the body Ψ in hyperelastic material. The classical principle of the minimum potential energy can be reformulated based on this strain energy. In the case of elastostatics, the potential is given by:

$$\Pi(\varphi) = \int_B \Psi dV - \int_B \mathbf{f}_b \cdot \varphi dV - \int_{\partial B_t} \mathbf{t} \cdot \varphi dA \quad (50)$$

To find a stationary point, we minimize the potential energy:

$$\min_{\varphi \in H} \Pi(\varphi) \quad (51)$$

after discretization of domain, where H is the space of admissible (trial) functions. For analytical calculation of the integral inside each element, the fourth-order Taylor series has been used instead of log function, taking advantage of its simplicity.

To exemplify this problem, we considered plane strain condition in a beam problem with length $L = 4$ m and height $H = 1$ m and arbitrary traction load at the right end and is clamped at the left side. The goal is to map arbitrary traction into displacement fields.

In other words, the networks are employed to solve a range of problems with different arbitrary traction functions. The ground truth solution is obtained by FEniCS [59] on a fine mesh. The displacement field was computed using $n_x \times n_y$ uniformly spaced points within the domain, with $n_x = 200$ and $n_y = 50$. The optimization process utilized the Adam optimizer, and the network underwent training on 500 sets of GRF functions and was subsequently tested on 50 additional sets of GRF functions.

Figs. 8b and 8c display the convergence of the loss function, beside the box plot, which compares the distribution of the relative L^2 -error for training and test dataset. The median error is represented by the blue line within each box, while the box bounds represent the interquartile range. This comparison helps assess the consistency of model performance on both datasets. Moreover, the right plot shows the training and testing loss values across 1000 iterations. The blue line represents the training loss, and the orange line represents the testing loss. The initial steep decline indicates that the model quickly reduces its loss, followed by convergence to a steady state as training progresses. The stability in both curves at later iterations indicates that the model has reached a stable level of performance with minimal fluctuations. In addition, Fig. 8d shows the result of the VINO in predicting the displacement field of a sample from the test dataset. The VINO result has a very good agreement with the ground truth solution. The computational time taken for the training part is 1282 s, but once the training is complete, solving the problem with any arbitrary function takes approximately 0.07 s, compared to 40 s on average for FEniCS.

3.2.3. Plate with arbitrary voids

As a third practical example, we consider parametric partial differential equations like Eq. (40) but on various domains. Assume the problem domain Ω_p is parameterized by design parameters $p \in P$, which follows some distribution $p \sim \mu$. Therefore, the problem setting becomes:

$$\begin{aligned} \frac{\partial \sigma_{xx}}{\partial x} + \frac{\partial \sigma_{xy}}{\partial y} + f_x &= 0 & \text{for } x, y \in \Omega_p \\ \frac{\partial \sigma_{yx}}{\partial x} + \frac{\partial \sigma_{yy}}{\partial y} + f_y &= 0 & \text{for } x, y \in \Omega_p \\ \mathbf{u}(x, y) &= \hat{\mathbf{u}} & \text{for } x = 0 \\ \boldsymbol{\sigma} \cdot \mathbf{n} &= \hat{\mathbf{t}}(y) & \text{for } x = L \end{aligned} \quad (52)$$

where $\hat{\mathbf{u}} \in \mathcal{U}$ is Dirichlet boundary condition, the $\hat{\mathbf{t}} \in \mathcal{T}$ is the Neumann boundary condition and the goal is approximation mapping $\mathcal{G}$ from problem domain to the PDE solution, $\Omega_p \mapsto u$. In this example, we assumed Dirichlet and Neumann boundary conditions are fixed and the domain space is given as meshes (uniform grid) $p^{(i)} = \{x^{(I)}, y^{(I)}\} \subset \Omega_p$. We consider $\Omega_p = [0, 5]^2 - \mathcal{V}$, where $\mathcal{V}$ is composed of arbitrary shaped holes inside the region $[1, 4]^2$. For more clarification, some samples are depicted in Fig. 9a. The plate is clamped on the left edge and constant tension traction is applied on the right edge.

We generated 1000 training data and 200 test data and used an IGA solver for the ground truth solution. Note that the ground truth solutions are not used during the training process and are just for comparing the final result of the network. The displacement field was computed using $n_x \times n_y$ uniformly spaced points within the domain, with $n_x = n_y = 200$ and the optimization process utilized the Adam optimizer. Figs. 9b and 9c illustrate both the distribution of the relative L^2 error and the convergence of the loss function. The box plot the relative L^2 error for the training and test datasets, with the blue line inside each box representing the median error. The boxes capture the interquartile range, allowing for an evaluation of the model's error consistency across both datasets. The line plot depicts the training and testing loss values over 1000 iterations. The blue line corresponds to the training loss, and the orange line to the testing loss. The sharp initial drop indicates a rapid reduction in loss, followed by stabilization, suggesting that the model achieves a steady state with minimal fluctuation as training progresses. In addition, Fig. 9 shows the result of the VINO in predicting the displacement field of some samples from the training and test dataset. The VINO result has an acceptable agreement with the ground truth solution.

4. Conclusion

In this work, we introduced the Variational Physics-Informed Neural Operator (VINO) that overcomes the limitations of data-driven methods that rely on large datasets of paired input and output, which are often costly or unavailable. VINO's core innovation lies in its unique incorporation of the variational form of physical laws directly into the learning process by discretizing the domain and using shape functions to analytically handle derivatives and integrals. This approach allows VINO to learn operators solely from physical principles, a capability that positions it as a unique tool among neural operators. Our results demonstrate that VINO can outperform existing approaches like PINO, particularly as mesh resolution increases. Notably, while other methods exhibit decreasing accuracy or inconsistent performance as mesh size grows, VINO demonstrates convergent behavior in fine meshes, highlighting its robustness and scalability. In final, VINO suggests a better way to incorporate the physical laws into neural operators.

This work highlights the potential of VINO to significantly reduce the dependence on paired training data, making it a powerful tool for addressing large-scale nonlinear problems. Beyond its standalone capabilities, VINO can also complement traditional numerical solvers such as finite element methods. For example, VINO can act as an efficient surrogate model to accelerate simulations in scenarios where traditional solvers are computationally expensive. By learning and generalizing complex solution mappings from minimal data, VINO provides a scalable alternative that can be seamlessly integrated into FEM workflows, particularly for tasks involving rapid prototyping or real-time prediction.

In VINO, boundary conditions (both Dirichlet and Neumann) are integrated into the variational formulation via the boundary terms arising in the weak form. This avoids the need for explicit penalty methods or auxiliary loss terms commonly used in other

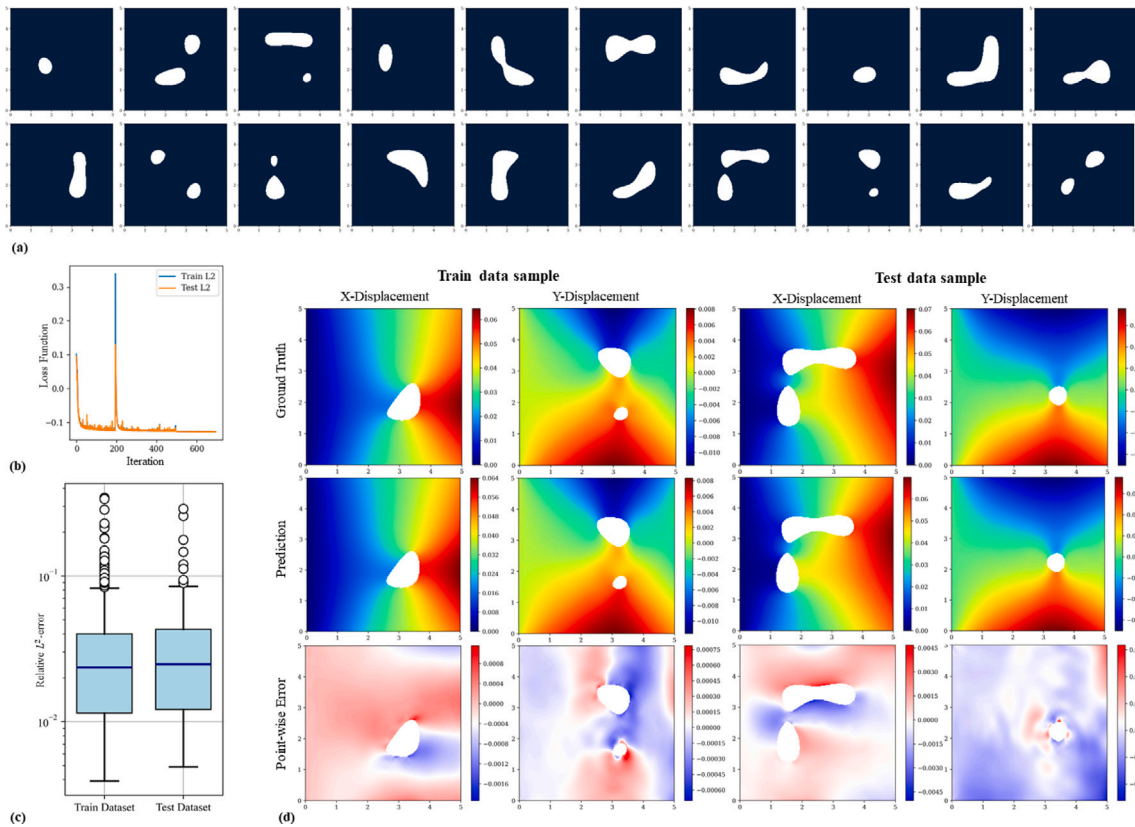


Fig. 9. Plate with arbitrary voids. (a) Random samples of allowable domain with arbitrary void shapes. (b) Loss function convergence over 1000 iterations for the train and test datasets, where both curves exhibit a rapid initial decrease followed by stabilization at lower values. (c) Box plots showing the relative L^2 -error distribution for the train and test datasets, with mean values highlighted in blue and standard deviations in red. The train dataset has a mean error of 0.035 with a standard deviation of 0.087, while the test dataset has a mean error of 0.032 with a standard deviation of 0.051. (d) Performance of VINO for predicting the displacement field of the porous material beam in a random sample from train and test dataset.

approaches. For Dirichlet boundary conditions, the network outputs are constructed to satisfy these conditions a priori. Neumann conditions, as natural boundary conditions, are directly incorporated into the energy functional, ensuring seamless and accurate enforcement.

For complex boundary geometries, element-wise discretization offers the flexibility to adapt to arbitrarily shaped domains by using standard mesh generation methods (e.g., triangular or tetrahedral elements). However, some modifications are needed to the FNO architecture to handle irregular geometries. VINO's ability to solve the problem of complex boundary geometries was discussed in Section 3.2.3, plate with arbitrary voids, where the voids were modeled by an immersed Cartesian mesh. Mixed boundary conditions (e.g., combinations of Dirichlet, Neumann, and Robin conditions) can be enforced simultaneously due to their natural emergence in the variational form of the PDE. Each type of condition contributes specific boundary integral terms to the variational functional, simplifying its implementation.

While VINO can be used for solving large-scale, nonlinear PDEs, it has limitations that make it less suitable in some scenarios. One of the primary limitations is its reliance on the variational formulation of the PDEs. This formulation can be particularly challenging when the PDEs do not admit a unique variational form. In such cases, where the PDEs arise from processes that lack conservative properties, VINO's framework may not be effectively applied. Furthermore, VINO's methodology works best when the nonlinearities of the system are relatively smooth. It may struggle in scenarios where the nonlinearity is discontinuous or exhibits complex behavior, such as shock waves in fluid dynamics or sudden transitions in material properties. Non-smooth behavior complicates the use of energy functionals and can lead to poor performance or convergence problems.

Currently, VINO is configured to solve PDEs with an available variational form. Future research should extend this approach to the weak form of equations, making it applicable to all types of PDEs. This study implemented VINO for rectangular elements, but the same methodology could be adapted for other element types, such as triangular or tetrahedron elements. Additionally, future work will refine VINO's architecture to handle more complex, multi-scale, and non-linear problems, explore adaptive meshing strategies, and expand applications to domains where traditional solvers encounter computational limits. VINO thus marks a promising advancement in neural operators for PDE learning, blending physics with machine learning to enhance operator learning frameworks. The development of a software library of pre-trained models also holds significant potential.

CRediT authorship contribution statement

Mohammad Sadegh Eshaghi: Writing – original draft, Validation, Software, Investigation, Formal analysis. **Cosmin Anitescu:** Writing – review & editing, Software, Methodology. **Manish Thombre:** Validation, Formal analysis. **Yizheng Wang:** Investigation, Conceptualization. **Xiaoying Zhuang:** Writing – review & editing, Conceptualization. **Timon Rabczuk:** Writing – review & editing, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The authors would like to acknowledge the support provided by the German Academic Exchange Service (DAAD) through a scholarship awarded to Mohammad Sadegh Eshaghi and by the Key Project of the National Natural Science Foundation of China (Grant No. 12332005) awarded to Yinghua Liu during this research, and the Compute-Servers of TU Ilmenau for providing computational resources.

Appendix A. Fourth-order equation using second-order shape function

In this study, we focused on first-order shape functions. However, the VINO framework can be readily extended to incorporate higher-order shape functions. This section demonstrates this capability by addressing a fourth-order problem described by the following equation:

$$k \frac{d^4 s(x)}{dx^4} = p(x), \quad x \in (0, 1), \quad (53)$$

with boundary conditions:

$$s(0) = s(1) = \frac{d^2 s}{dx^2}(0) = \frac{d^2 s}{dx^2}(1) = 0. \quad (54)$$

where $s(x)$ is the unknown solution, k is constant, and $p(x)$ is the ODE parameters. Here, the goal is to learn the operator $\mathcal{G}$ mapping from PDE parameter $p(x)$ to the solution $s(x)$. The corresponding energy functional to minimize is:

$$\Pi = \int_0^1 \left[\frac{k}{2} \left(\frac{d^2 s}{dx^2} \right)^2 - s(x) p(x) \right] dx. \quad (55)$$

As calculating energy function involves second derivatives, using at least a second-order function is required, therefore, the shape functions N_1, N_2, N_3 defined over an element as:

$$N_1(x) = 1 - 3\xi + 2\xi^2, \quad (56)$$

$$N_2(x) = 4\xi - 4\xi^2, \quad (57)$$

$$N_3(x) = -\xi + 2\xi^2, \quad (58)$$

where $\xi = x/a$ and a is the element length, the solution within the element $\hat{s}_e(x)$ can be expressed in terms of nodal values $\tilde{s}_1^e, \tilde{s}_2^e, \tilde{s}_3^e$:

$$\mathcal{G}_\theta^e(p) = s^e \approx \hat{s}^e = \sum_{a=1}^3 N_a \tilde{s}_a = \mathbf{N} \tilde{\mathbf{s}} \quad (59)$$

Then we can calculate the second derivative and the integral analytically, so the functional can be expressed:

$$\Pi = \sum_e \left[\frac{k}{8a^3} (\tilde{s}_1^e - 2\tilde{s}_2^e + \tilde{s}_3^e)^2 - \tilde{\mathbf{s}}_e^T \mathbf{M}^e \tilde{\mathbf{f}}_e \right]. \quad (60)$$

where $\mathbf{M}^e = \int_{\Omega_e} \mathbf{N}^T \mathbf{N}$ is a scalar quantity.

Finally, for a collection of input–output pairs $\{s^{(i)}, \mathbf{p}^{(i)}\}$, where $s^{(i)} = \mathcal{G}(\mathbf{p}^{(i)})$, the loss function for FNO is the same as Eq. (29) and PINO and VINO loss function is defined as

$$\mathcal{L}_p = \frac{1}{n} \sum_{i=1}^n \left\| k \frac{d^4}{dx^4} \mathcal{G}_\theta(\mathbf{p}^{(i)})(x) - \mathbf{p}^{(i)}(x) \right\|_{L^2}^2 \quad (61)$$

$$\mathcal{L}_v = \frac{1}{n} \sum_{i=1}^n \sum_e \left[\frac{k}{8a^3} (\tilde{s}_1^e - 2\tilde{s}_2^e + \tilde{s}_3^e)^2 - \tilde{\mathbf{s}}_e^T \mathbf{M}^e \tilde{\mathbf{f}}_e \right] \quad (62)$$

In this case, $\mathbf{p}^{(i)} = [p^{(i)}(x_1), p^{(i)}(x_2), \dots, p^{(i)}(x_m)]$ and $\mathbf{s}^{(i)} = [s^{(i)}(x_1), s^{(i)}(x_2), \dots, s^{(i)}(x_m)]$, with $m = 64$ grid points in the $[0, 1]$ domain, and $k = 1/12$. The input functions $p(x)$ are sampled from a Gaussian random field (GRF). We use $n = 1,000$ training samples and

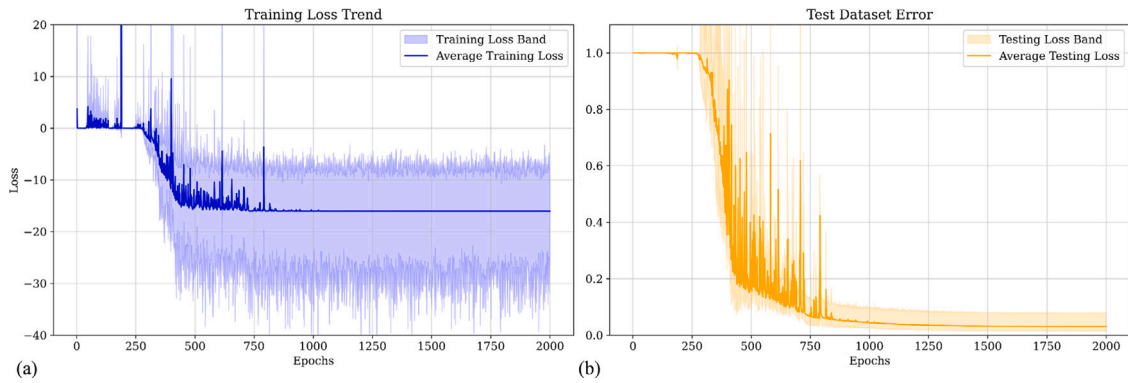


Fig. 10. Fourth-Order Eq. (a) Training Loss Trend showing the loss band across epochs and the average training loss. **(b)** Testing Dataset Error illustrating the test loss band and the average testing loss over epochs.

generate an additional test dataset of 100 different random input functions and solutions and train the models using gradient descent with the Adam optimizer.

The numerical setup involved sampling $p(x)$ from a Gaussian random field evaluated over a grid of $m = 64$ points in the interval $[0, 1]$ with the Adam optimizer for 2000 iterations. To facilitate the retrieval of our results, we have listed all the hyperparameters in Table 2. The results in Fig. 10 highlight the loss trends for both the training and testing dataset. Subfigure (a) presents the training loss trend, with the shaded region representing the variability of loss values across individual batches in each epoch (loss band) and the solid line indicating the average training loss. This trend demonstrates a steady decrease in training loss over the epochs, eventually stabilizing, suggesting that the model effectively minimizes the energy functional and learns the mapping $\mathcal{G}$ from the PDE parameter $p(x)$ to the solution $s(x)$. Subfigure (b) shows the testing dataset error during training process, where the test loss band (shaded region) captures the variability in loss over the test dataset, while the solid line represents the average test loss. The test loss exhibits a gradual decrease, reflecting the generalization capability of the VINO framework to unseen data. Notably, the error of the model in predicting the test dataset converges to $0.52 \pm 0.69\%$, indicating a stable and consistent prediction performance.

Learning this equation using PINO involves calculating fourth-order derivatives of the solution $s(x)$, which requires significant computational effort and introduces considerable error in the approximation. The challenge lies in the complexity of the fourth-order derivatives, which must be computed numerically at each step. However, with the VINO strategy, this problem is mitigated by reducing the derivative order to second-order derivatives and by analytically computing the second derivatives, the functional can be expressed more efficiently and with fewer errors. This approach facilitates solving the equation with very high accuracy, without the need for any labeled data, thus showcasing the significant advantage of the VINO approach in terms of computational efficiency and precision.

Appendix B. Discussion on the effect of hyperparameters in VINO

In this section, we investigate the effect of various hyperparameters on the performance of the VINO for learning parametric second-order antiderivative, and we limit this discussion to this specific problem for the sake of brevity. Specifically, we focus on the influence of **batch size**, **learning rate**, **decay factor (gamma)**, **Fourier modes number**, **Fourier layer width**, and **number of layers**. Each of these hyperparameters plays a significant role in determining the training efficiency, accuracy, and overall performance of the VINO model in solving PDEs. When considering the effect of one hyperparameter, the others are fixed to the values in Table 2.

- **Batch Size:** The batch size is an essential hyperparameter that controls the number of samples used in each iteration of the training process. The effect of batch size on VINO's performance is shown in the accompanying box plot (Fig. 11a), which displays the relative error across different batch sizes. As observed, larger batch sizes tend to result in smaller values in the final testing errors.
- **Learning Rate:** The learning rate controls the step size during the optimization process. An appropriately chosen learning rate allows the model to converge more quickly and avoid overshooting the optimal solution. As indicated by the data (Fig. 11b), lower learning rates result in stable convergence but require many epochs to achieve minimal error. Higher learning rates allow for faster convergence but may cause larger oscillations in the error values before reaching the final solution. In this case learning rate equals 0.0005 shows the best performance.
- **Decay Factor (Gamma):** The decay factor, often referred to as gamma, controls the rate at which the learning rate is reduced during training. The influence of gamma on VINO's accuracy is presented in the corresponding box plot (Fig. 11c). We observe that higher gamma values cause quicker adjustments in learning rate, potentially speeding up the initial phase of training but may result in a less refined solution. Lower gamma values, on the other hand, lead to slower reductions in learning rate and can result in more accurate solutions due to the extended exploration of the parameter space.

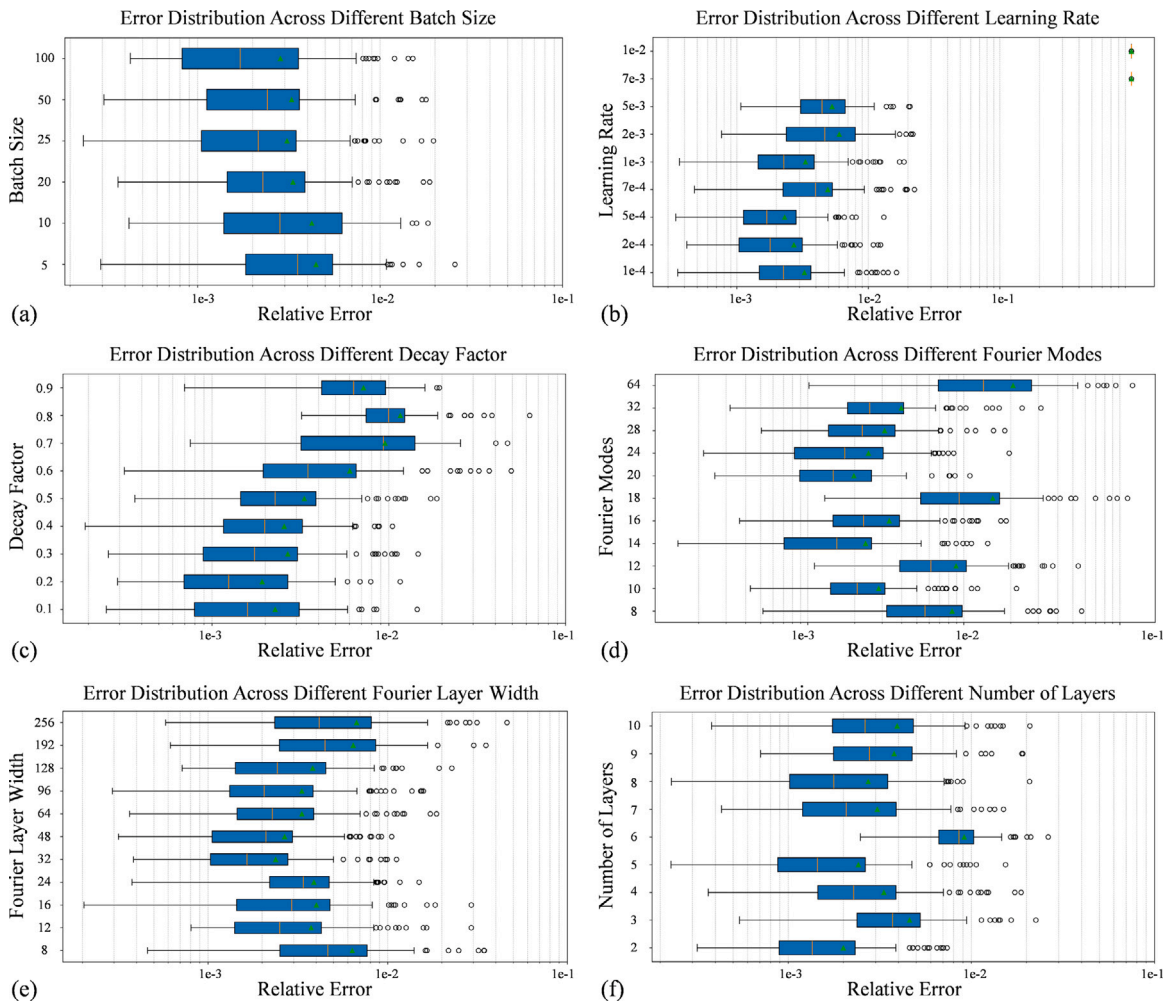


Fig. 11. Hyperparameters impact on test dataset error.

- **Fourier Modes:** The number of Fourier modes in the neural network represents how much frequency information is incorporated into the model's representations. Using more modes allows the model to capture a broader range of variations in the solution, making it better equipped to handle more complex physical processes represented by PDEs. However, more modes also increase the computational burden and can potentially introduce overfitting to the data if not controlled appropriately. Fig. 11d shows the performance of VINO with different numbers of Fourier modes space. In our experiment, 14 modes demonstrated better performance in comparison with other values.
- **Fourier Layer Width:** The width of the Fourier layers refers to the number of neurons in each Fourier layer. From the experiments (Fig. 11e), we observe that broader networks improve the model's predictive accuracy, but only to a certain extent. Beyond a certain threshold, which in our experiment is 32, the accuracy plateaus, suggesting that further increases in width provide diminishing returns and also simply contribute to increased computational costs.
- **Number of Layers:** The number of layers in the network influences the model's capacity to represent deeper hierarchical features of the input data. The box plot for the number of layers (Fig. 11f) illustrates that deeper networks (i.e., models with more layers) generally achieve higher relative errors, and they also take longer to train and may require additional regularization techniques, because the number of the training dataset is fixed.

Data availability

The code and data supporting this work are publicly available at <https://github.com/eshaghi-ms/VINO>.

References

- [1] M. Raissi, P. Perdikaris, G.E. Karniadakis, Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, *J. Comput. Phys.* 378 (2019) 686–707, <http://dx.doi.org/10.1016/j.jcp.2018.10.045>.
- [2] B. Yu, W. E, The deep Ritz method: a deep learning-based numerical algorithm for solving variational problems, *Commun. Math. Stat.* 6 (1) (2018) 1–12, <http://dx.doi.org/10.1007/s40304-018-0127-z>.
- [3] E. Samaniego, C. Anitescu, S. Goswami, V. Nguyen-Thanh, H. Guo, K. Hamdia, X. Zhuang, T. Rabczuk, An energy approach to the solution of partial differential equations in computational mechanics via machine learning: Concepts, implementation, and applications, *Comput. Methods Appl. Mech. Engrg.* 362 (2020) 112790, <http://dx.doi.org/10.1016/j.cma.2019.112790>.
- [4] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, A. Stuart, K. Bhattacharya, A. Anandkumar, Multipole graph neural operator for parametric partial differential equations, *Adv. Neural Inf. Process. Syst.* 33 (2020) 6755–6766, <http://dx.doi.org/10.48550/arXiv.2006.09535>.
- [5] L. Lu, P. Jin, G. Pang, Z. Zhang, G.E. Karniadakis, Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators, *Nat. Mach. Intell.* 3 (3) (2021) 218–229, <http://dx.doi.org/10.1038/s42256-021-00302-5>.
- [6] Z. Li, H. Zheng, N. Kovachki, D. Jin, H. Chen, B. Liu, K. Azizzadenesheli, A. Anandkumar, Physics-informed neural operator for learning partial differential equations, *ACM/JMS J. Data Sci.* 1 (3) (2024) 1–27, <http://dx.doi.org/10.1145/3648506>.
- [7] S. Wang, H. Wang, P. Perdikaris, Learning the solution operator of parametric partial differential equations with physics-informed DeepONets, *Sci. Adv.* 7 (40) (2021) eabi8605, <http://dx.doi.org/10.1126/sciadv.abi8605>.
- [8] S.L. Brunton, B.R. Noack, P. Koumoutsakos, Machine learning for fluid mechanics, *Annu. Rev. Fluid Mech.* 52 (1) (2020) 477–508, <http://dx.doi.org/10.1146/annurev-fluid-010719-060214>.
- [9] M.S. Eshaghi, M. Bamdad, C. Anitescu, Y. Wang, X. Zhuang, T. Rabczuk, Applications of scientific machine learning for the analysis of functionally graded porous beams, *Neurocomputing* 619 (2025) 129119, <http://dx.doi.org/10.1016/j.neucom.2024.129119>.
- [10] C. Kaewnuratchadasorn, J. Wang, C.-W. Kim, Physics-informed neural operator solver and super-resolution for solid mechanics, *Comput.-Aided Civ. Infrastruct. Eng.* (2024) <http://dx.doi.org/10.1111/mice.13292>.
- [11] M. Raissi, A. Yazdani, G.E. Karniadakis, Hidden fluid mechanics: Learning velocity and pressure fields from flow visualizations, *Science* 367 (6481) (2020) 1026–1030, <http://dx.doi.org/10.1126/science.aaw4741>.
- [12] A.M. Tartakovsky, C.O. Marrero, P. Perdikaris, G.D. Tartakovsky, D. Barajas-Solano, Physics-informed deep neural networks for learning parameters and constitutive relationships in subsurface flow problems, *Water Resour. Res.* 56 (5) (2020) <http://dx.doi.org/10.1029/2019WR026731>, e2019WR026731.
- [13] X. Lu, Y. Wang, Surrogate modeling for radiative heat transfer using physics-informed deep neural operator networks, *Proc. Combust. Inst.* 40 (1–4) (2024) 105282, <http://dx.doi.org/10.1016/j.proci.2024.105282>.
- [14] G. Kissas, Y. Yang, E. Hwuang, W.R. Witschey, J.A. Detre, P. Perdikaris, Machine learning in cardiovascular flows modeling: Predicting arterial blood pressure from non-invasive 4D flow MRI data using physics-informed neural networks, *Comput. Methods Appl. Mech. Engrg.* 358 (2020) 112623, <http://dx.doi.org/10.1016/j.cma.2019.112623>.
- [15] F. Sahli Costabal, Y. Yang, P. Perdikaris, D.E. Hurtado, E. Kuhl, Physics-informed neural networks for cardiac activation mapping, *Front. Phys.* 8 (2020) 42, <http://dx.doi.org/10.3389/fphy.2020.00042>.
- [16] L. Lu, M. Dao, P. Kumar, U. Ramamurty, G.E. Karniadakis, S. Suresh, Extraction of mechanical properties of materials through deep learning from instrumented indentation, *Proc. Natl. Acad. Sci.* 117 (13) (2020) 7052–7062, <http://dx.doi.org/10.1073/pnas.1922210117>.
- [17] Y. Chen, L. Lu, G.E. Karniadakis, L. Dal Negro, Physics-informed neural networks for inverse problems in nano-optics and metamaterials, *Opt. Express* 28 (8) (2020) 11618–11633, <http://dx.doi.org/10.1364/OE.384875>.
- [18] D. Elbrächter, P. Grohs, A. Jentzen, C. Schwab, DNN expression rate analysis of high-dimensional PDEs: application to option pricing, *Constr. Approx.* 55 (1) (2022) 3–71, <http://dx.doi.org/10.1007/s00365-021-09541-6>.
- [19] J. Han, A. Jentzen, W. E, Solving high-dimensional partial differential equations using deep learning, *Proc. Natl. Acad. Sci.* 115 (34) (2018) 8505–8510, <http://dx.doi.org/10.1073/pnas.1718942115>.
- [20] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, A. Anandkumar, Fourier neural operator for parametric partial differential equations, 2020, <http://dx.doi.org/10.48550/arXiv.2010.08895>, arXiv preprint [arXiv:2010.08895](http://arxiv.org/abs/2010.08895).
- [21] L. Lu, P. Jin, G. Pang, Z. Zhang, G.E. Karniadakis, Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators, *Nat. Mach. Intell.* 3 (3) (2021) 218–229, <http://dx.doi.org/10.1038/s42256-021-00302-5>.
- [22] B. Liu, N. Kovachki, Z. Li, K. Azizzadenesheli, A. Anandkumar, A.M. Stuart, K. Bhattacharya, A learning-based multiscale method and its application to inelastic impact problems, *J. Mech. Phys. Solids* 158 (2022) 104668, <http://dx.doi.org/10.1016/j.jmps.2021.104668>.
- [23] H. Yang, Z. Li, K. Sastry, S. Mukhopadhyay, M. Kilgard, A. Anandkumar, B. Khailany, V. Singh, H. Ren, Generic lithography modeling with dual-band optics-inspired neural networks, in: *Proceedings of the 59th ACM/IEEE Design Automation Conference*, 2022, pp. 973–978, <http://dx.doi.org/10.1145/3489517.3530580>.
- [24] G. Wen, Z. Li, Q. Long, K. Azizzadenesheli, A. Anandkumar, S.M. Benson, Real-time high-resolution CO₂ geological storage prediction using nested Fourier neural operators, *Energy Environ. Sci.* 16 (4) (2023) 1732–1741, <http://dx.doi.org/10.1039/D2EE04204E>.
- [25] B. Bonev, T. Kurth, C. Hundt, J. Pathak, M. Baust, K. Kashinath, A. Anandkumar, Spherical Fourier neural operators: Learning stable dynamics on the sphere, in: A. Krause, E. Brunskill, K. Cho, B. Engelhardt, S. Sabato, J. Scarlett (Eds.), *Proceedings of the 40th International Conference on Machine Learning*, in: *Proceedings of Machine Learning Research*, vol. 202, PMLR, 2023, pp. 2806–2823, <http://dx.doi.org/10.48550/arXiv.2306.03838>.
- [26] H. Hersbach, B. Bell, P. Berrisford, S. Hirahara, A. Horányi, J. Muñoz-Sabater, J. Nicolas, C. Peubey, R. Radu, D. Schepers, et al., The ERA5 global reanalysis, *Q. J. R. Meteorol. Soc.* 146 (730) (2020) 1999–2049, <http://dx.doi.org/10.1002/qj.3803>.
- [27] J.Y. Lee, S. Ko, Y. Hong, Finite element operator network for solving parametric pdes, 2023, <http://dx.doi.org/10.48550/arXiv.2308.04690>.
- [28] D. Kochkov, J.A. Smith, A. Alieva, Q. Wang, M.P. Brenner, S. Hoyer, Machine learning-accelerated computational fluid dynamics, *Proc. Natl. Acad. Sci.* 118 (21) (2021) e2101784118, <http://dx.doi.org/10.1073/pnas.2101784118>.
- [29] J. Pathak, M. Mustafa, K. Kashinath, E. Motheau, T. Kurth, M. Day, ML-PDE: A Framework for a Machine Learning Enhanced PDE Solver, in: *APS March Meeting Abstracts*, in: *APS Meeting Abstracts*, vol. 2021, 2021, p. J25.002.
- [30] D. Greenfield, M. Galun, R. Basri, I. Yavneh, R. Kimmel, Learning to optimize multigrid PDE solvers, in: K. Chaudhuri, R. Salakhutdinov (Eds.), *Proceedings of the 36th International Conference on Machine Learning*, in: *Proceedings of Machine Learning Research*, vol. 97, PMLR, 2019, pp. 2415–2423.
- [31] A. Mishra, C. Anitescu, P.R. Budarapu, S. Natarajan, P.R. Vundavilli, T. Rabczuk, An artificial neural network based deep collocation method for the solution of transient linear and nonlinear partial differential equations, *Front. Struct. Civ. Eng.* 18 (8) (2024) 1296–1310, <http://dx.doi.org/10.1007/s11709-024-1011-4>.
- [32] J. Sirignano, K. Spiliopoulos, DGM: A deep learning algorithm for solving partial differential equations, *J. Comput. Phys.* 375 (2018) 1339–1364, <http://dx.doi.org/10.1016/j.jcp.2018.08.029>.
- [33] S. Wang, X. Yu, P. Perdikaris, When and why PINNs fail to train: A neural tangent kernel perspective, *J. Comput. Phys.* 449 (2022) 110768, <http://dx.doi.org/10.1016/j.jcp.2021.110768>.
- [34] O. Fuks, H.A. Tchelepi, Limitations of physics informed machine learning for nonlinear two-phase transport in porous media, *J. Mach. Learn. Model. Comput.* 1 (1) (2020) <http://dx.doi.org/10.1615/JMachLearnModelComput.2020033905>.

- [35] A.J. Huang, S. Agarwal, On the limitations of physics-informed deep learning: Illustrations using first-order hyperbolic conservation law-based traffic flow models, *IEEE Open J. Intell. Transp. Syst.* 4 (2023) 279–293, <http://dx.doi.org/10.1109/OJITS.2023.3268026>.
- [36] Y. Wang, J. Sun, W. Li, Z. Lu, Y. Liu, CENN: Conservative energy method based on neural networks with subdomains for solving variational problems involving heterogeneous and complex geometries, *Comput. Methods Appl. Mech. Engrg.* 400 (2022) 115491, <http://dx.doi.org/10.1016/j.cma.2022.115491>.
- [37] W. Li, M.Z. Bazant, J. Zhu, A physics-guided neural network framework for elastic plates: Comparison of governing equations-based and energy-based approaches, *Comput. Methods Appl. Mech. Engrg.* 383 (2021) 113933, <http://dx.doi.org/10.1016/j.cma.2021.113933>.
- [38] H. Sheng, C. Yang, PFNN: A penalty-free neural network method for solving a class of second-order boundary-value problems on complex geometries, *J. Comput. Phys.* 428 (2021) 110085, <http://dx.doi.org/10.1016/j.jcp.2020.110085>.
- [39] O.C. Zienkiewicz, R.L. Taylor, *The Finite Element Method: Its Basis and Fundamentals*, Elsevier, 2005, <http://dx.doi.org/10.1016/C2009-0-24909-9>.
- [40] A.D. Jagtap, E. Kharazmi, G.E. Karniadakis, Conservative physics-informed neural networks on discrete domains for conservation laws: Applications to forward and inverse problems, *Comput. Methods Appl. Mech. Engrg.* 365 (2020) 113028, <http://dx.doi.org/10.1016/j.cma.2020.113028>.
- [41] S. Goswami, A. Bora, Y. Yu, G.E. Karniadakis, Physics-informed deep neural operator networks, in: T. Rabczuk, K.-J. Bathe (Eds.), *Machine Learning in Modeling and Simulation: Methods and Applications*, Springer International Publishing, Cham, 2023, pp. 219–254, http://dx.doi.org/10.1007/978-3-031-36644-4_6.
- [42] S. Goswami, M. Yin, Y. Yu, G.E. Karniadakis, A physics-informed variational DeepONet for predicting crack path in quasi-brittle materials, *Comput. Methods Appl. Mech. Engrg.* 391 (2022) 114587, <http://dx.doi.org/10.1016/j.cma.2022.114587>.
- [43] J. Bradbury, R. Frostig, P. Hawkins, M.J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, Q. Zhang, *JAX: composable transformations of Python+NumPy programs*, 2018.
- [44] C.K. Williams, C.E. Rasmussen, *Gaussian Processes for Machine Learning*, vol. 2, (no. 3) MIT press Cambridge, MA, 2006, <http://dx.doi.org/10.7551/mitpress/3206.001.0001>.
- [45] D.P. Kingma, Adam: A method for stochastic optimization, 2014, <http://dx.doi.org/10.48550/arXiv.1412.6980>.
- [46] L. Lu, X. Meng, S. Cai, Z. Mao, S. Goswami, Z. Zhang, G.E. Karniadakis, A comprehensive and fair comparison of two neural operators (with practical extensions) based on fair data, *Comput. Methods Appl. Mech. Engrg.* 393 (2022) 114778, <http://dx.doi.org/10.1016/j.cma.2022.114778>.
- [47] H. You, Y. Yu, M. D'Elia, T. Gao, S. Silling, Nonlocal kernel network (NKN): A stable and resolution-independent deep neural network, *J. Comput. Phys.* 469 (2022) 111536, <http://dx.doi.org/10.1016/j.jcp.2022.111536>.
- [48] H. You, Q. Zhang, C.J. Ross, C.-H. Lee, Y. Yu, Learning deep implicit Fourier neural operators (IFNOs) with applications to heterogeneous material modeling, *Comput. Methods Appl. Mech. Engrg.* 398 (2022) 115296, <http://dx.doi.org/10.1016/j.cma.2022.115296>.
- [49] Y. Li, Z. Feng, L. Hao, L. Huang, C. Xin, Y. Wang, E. Bilotti, K. Essa, H. Zhang, Z. Li, et al., A review on functionally graded materials and structures via additive manufacturing: from multi-scale design to versatile functional properties, *Adv. Mater. Technol.* 5 (6) (2020) 1900981, <http://dx.doi.org/10.1002/admt.201900981>.
- [50] L. Audouard, P. Bertrand, J. Justin, C. Langlade, L. Vingert, M. Tsoutsouva, M. Garcia, A. Julian-Jankowiak, Resistance of ceramic/metal functionally graded materials in the flame of a combustion chamber under harsh thermal and environmental conditions, *Mater. Charact.* 208 (2024) 113616, <http://dx.doi.org/10.1016/j.matchar.2023.113616>.
- [51] M. Babaei, F. Kiarasi, K. Asemi, M. Hosseini, Functionally graded saturated porous structures: A review, *J. Comput. Appl. Mech.* 53 (2) (2022) 297–308, <http://dx.doi.org/10.22059/JCAMECH.2022.342710.719>.
- [52] P.M. Ramteke, S.K. Panda, Computational modelling and experimental challenges of linear and nonlinear analysis of porous graded structure: a comprehensive review, *Arch. Comput. Methods Eng.* 30 (5) (2023) 3437–3452, <http://dx.doi.org/10.1007/s11831-023-09908-x>.
- [53] D. Chen, J. Yang, S. Kitipornchai, Free and forced vibrations of shear deformable functionally graded porous beams, *Int. J. Mech. Sci.* 108 (2016) 14–22, <http://dx.doi.org/10.1016/j.ijmecsci.2016.01.025>.
- [54] F. Kiarasi, M. Babaei, P. Sarvi, K. Asemi, M. Hosseini, M. Omid Bidgoli, A review on functionally graded porous structures reinforced by graphene platelets, *J. Comput. Appl. Mech.* 52 (4) (2021) 731–750, <http://dx.doi.org/10.22059/JCAMECH.2021.335739.675>.
- [55] D. Chen, K. Gao, J. Yang, L. Zhang, Functionally graded porous structures: Analyses, performances, and applications—A Review, *Thin-Walled Struct.* 191 (2023) 111046, <http://dx.doi.org/10.1016/j.tws.2023.111046>.
- [56] S. Agarwal, A. Chakraborty, S. Gopalakrishnan, Large deformation analysis for anisotropic and inhomogeneous beams using exact linear static solutions, *Compos. Struct.* 72 (1) (2006) 91–104, <http://dx.doi.org/10.1016/j.compstruct.2004.10.019>.
- [57] C. Anitescu, M.N. Hossain, T. Rabczuk, Recovery-based error estimation and adaptivity using high-order splines over hierarchical T-meshes, *Comput. Methods Appl. Mech. Engrg.* 328 (2018) 638–662, <http://dx.doi.org/10.1016/j.cma.2017.08.032>.
- [58] D. Hendrycks, K. Gimpel, Gaussian error linear units (GELUs), 2016, <http://dx.doi.org/10.48550/arXiv.1606.08415>.
- [59] I.A. Baratta, J.P. Dean, J.S. Dokken, M. Habera, J.S. Hale, C.N. Richardson, M.E. Rognes, M.W. Scroggs, N. Sime, G.N. Wells, DOLFINx: the next generation FEniCS problem solving environment, 2023, <http://dx.doi.org/10.5281/zenodo.10447666>, preprint.